“시간이 아깝지 않은”

SK네트웍스 Family AI캠프 교재4

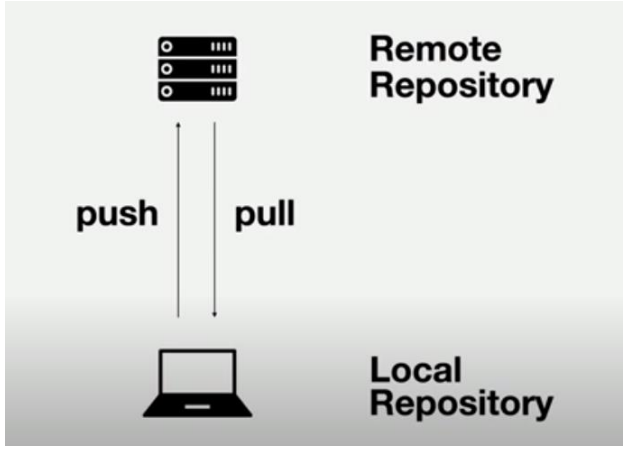
강사 이영배

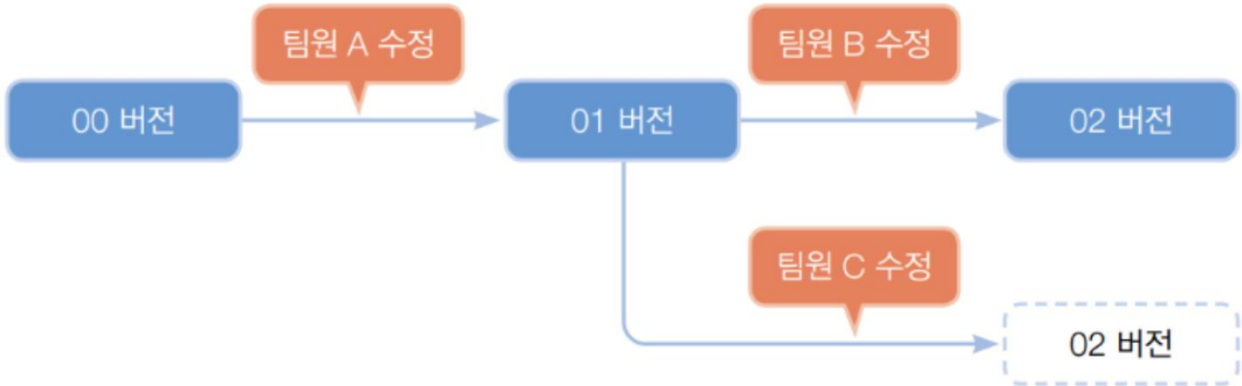
06. Git과 GitHub를 이용한 형상관리

github를 이용한 코드(파일) 저장 및 버전 관리

- 코드 버전 관리의 필요성
  1) 코드 개발 시 코드 수정이 빈번함
  2) 내가 원하는 시점(버전)으로 이동할 수 있게 해 주는 것이 버전 관리이며, 이를 도와주는 툴이 버전 관리 시스템 임
- git과 github
  1) git
     ① 로컬 컴퓨터에 설치하는 버전 관리 시스템
     ② 코드 및 파일의 변경 사항을 추적하고 이전 버전으로 되돌릴 수 있도록 함
  2) github
     ① 로컬 컴퓨터에 있는 git 저장소를 클라우드에 저장 및 코드 공유, 협업을 지원하는 웹 기반 플랫폼
     ② 대표적인 github : github.com

github를 이용한 코드 저장 및 버전 관리





github를 이용한 코드 저장 및 버전 관리

3) git과 github를 이용한 코드 저장 및 다운로드 절차

① git 설치: 내 컴퓨터에 버전 관리 도구 설치

② 로컬 저장소(local repository) 생성: 코드를 저장하고 관리할 폴더 설정

③ github.com 회원 가입, 원격 저장소(remote repository) 생성: 백업 및 협업 공간 마련

④ git 환경 설정: 내 이름과 이메일(github.com 회원 가입시 사용한 것) 저장 → 작업자 정보 설정

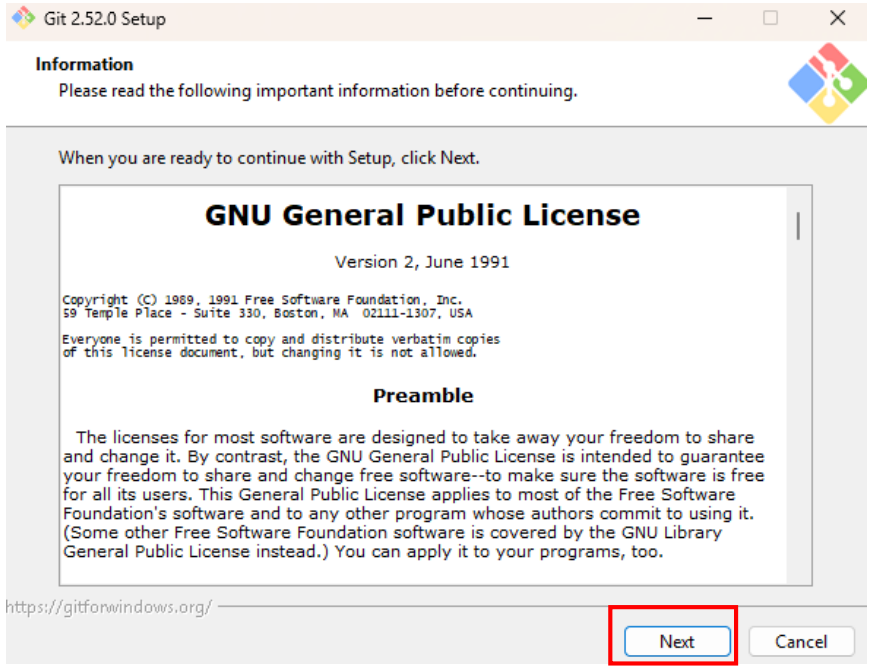
⑤ add & commit: 작업의 결과물을 선별, 로컬 저장소에 저장

⑥ 로컬 저장소와 원격 저장소 연결, 원격 저장소 업로드(push) 진행

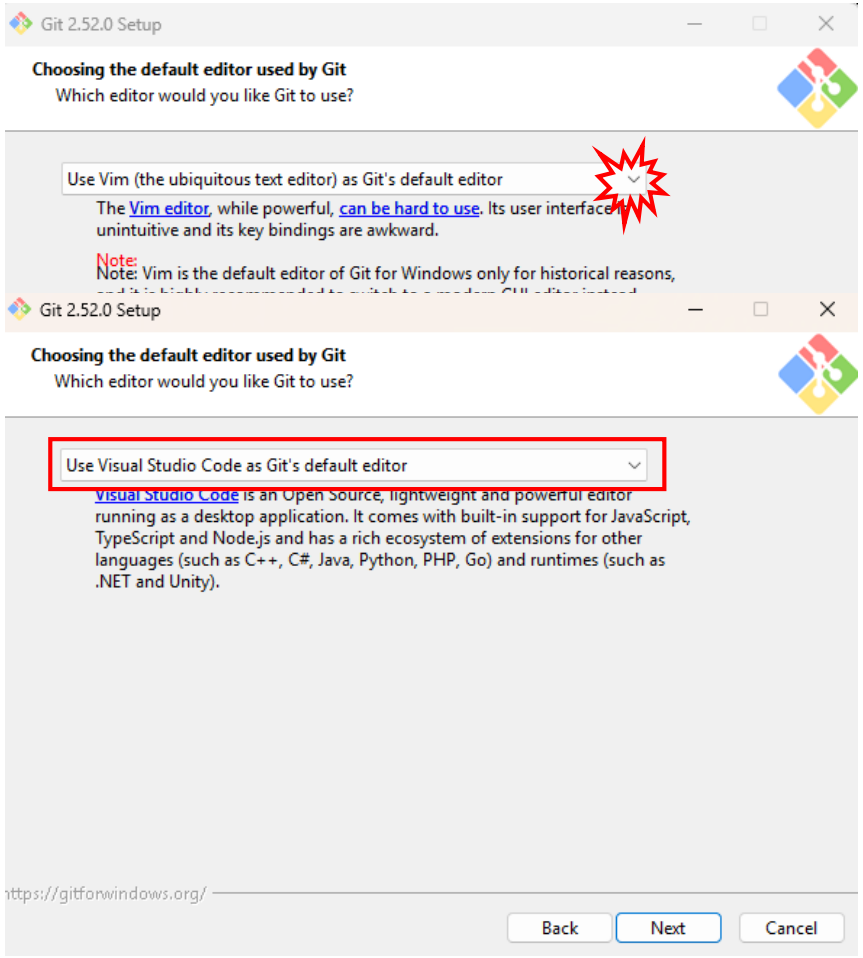
⑦ pull: 동료의 작업 결과물 다운로드

github를 이용한 코드 저장 및 버전 관리

4) git 설치(윈도우)



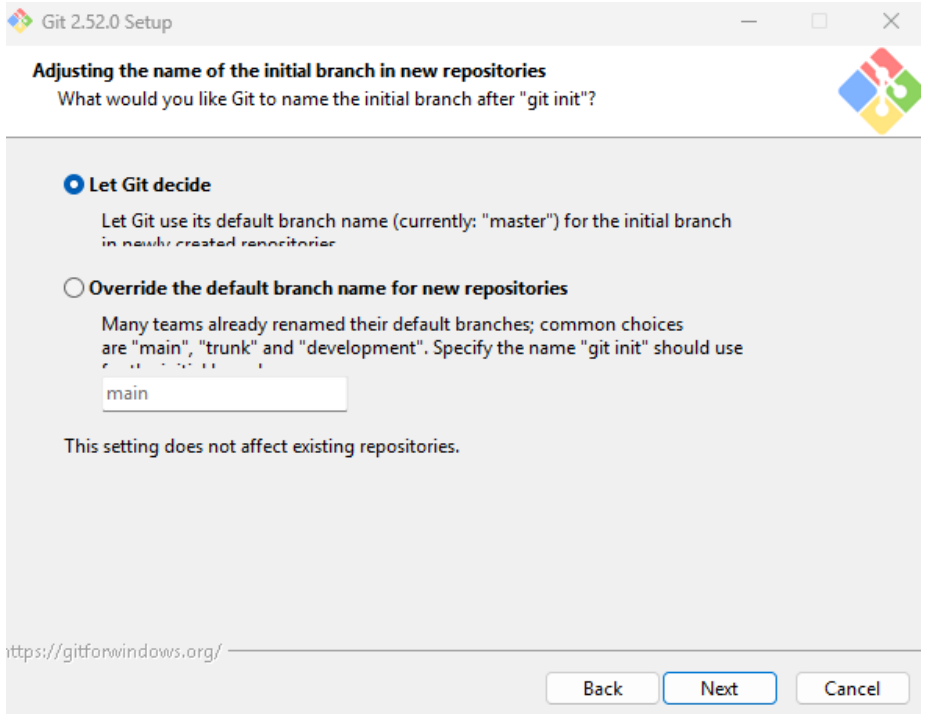
Next 클릭 4번 진행

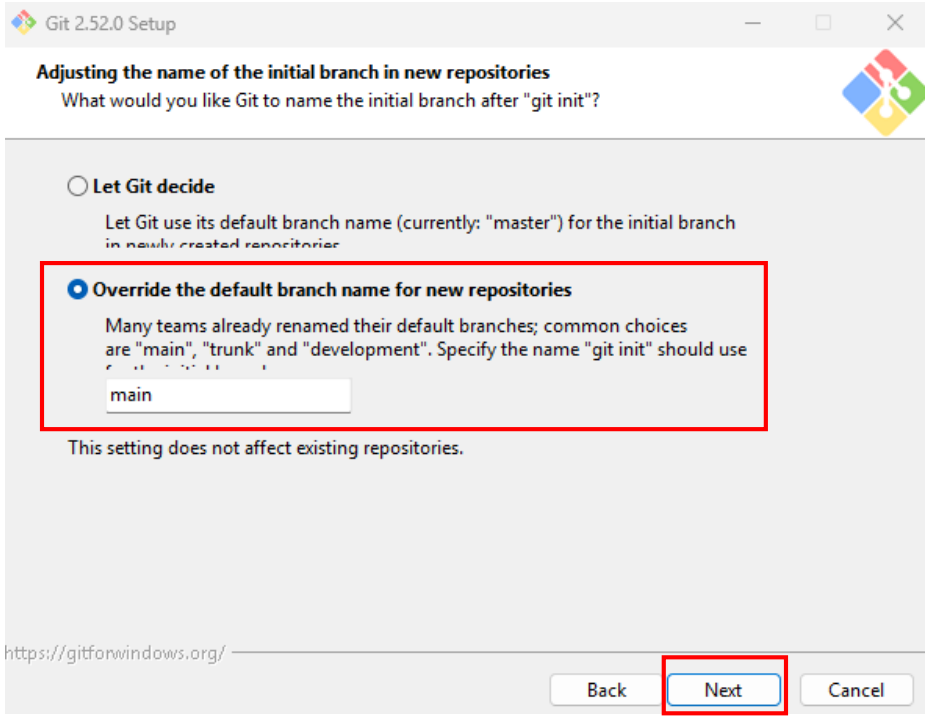


github를 이용한 코드 저장 및 버전 관리

4) git 설치(윈도우)

Next 클릭 5번 진행

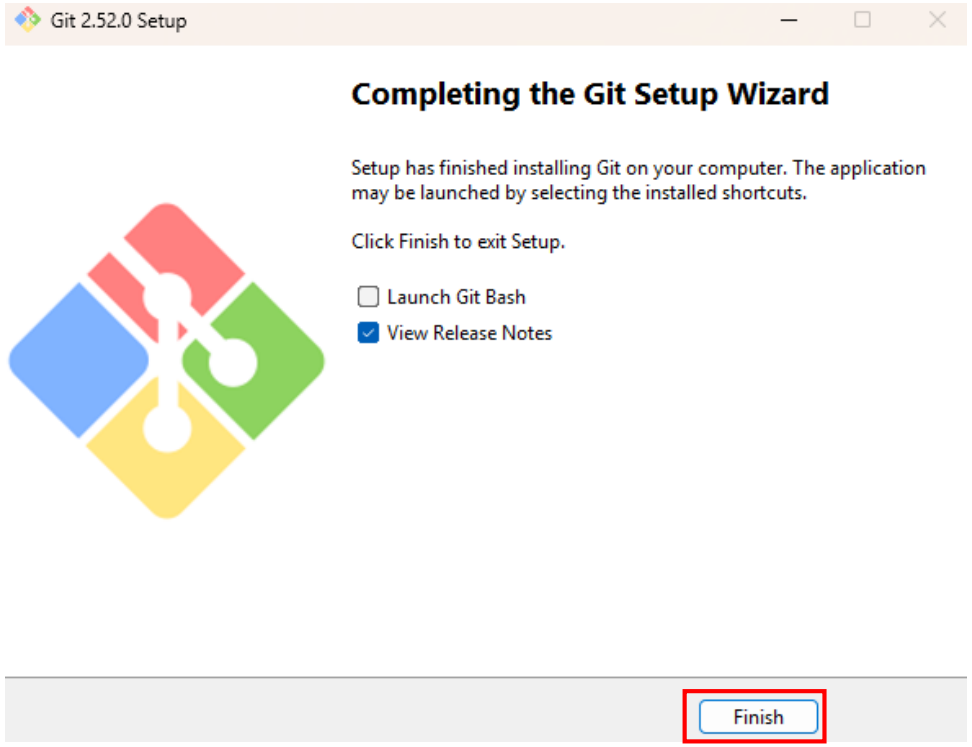




github를 이용한 코드 저장 및 버전 관리

4) git 설치(윈도우)

마지막 까지 Next 선택 및 진행



github를 이용한 코드 저장 및 버전 관리

4) git 설치(macOS)

① 맥에는 이미 Git 설치를 도와주는 도구가 내장되어 있음

② Command + Space를 눌러 스포트라이트를 켜고 "Terminal" (터미널) 실행

③ 아래 명령어를 입력하고 엔터를 누릅니다.

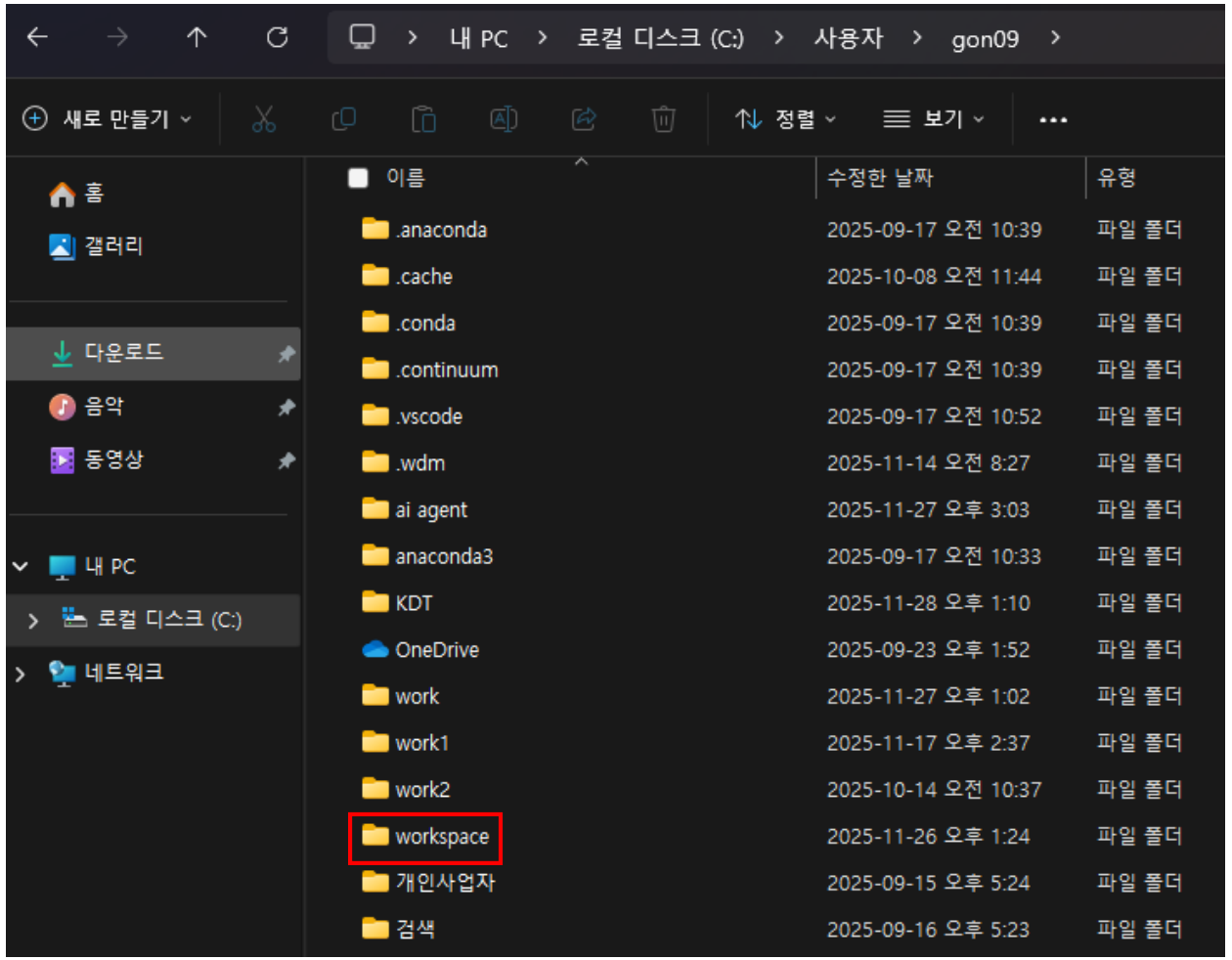
git --version

④ 만약 Git이 설치되어 있지 않다면, "명령어 라인 개발자 도구를 설치하시겠습니까?" 라는 팝업이 생성됨

⑤ [설치] 버튼을 누르고 동의하면 자동으로 설치가 진행됩니다. (시간이 조금 걸릴 수 있습니다.)

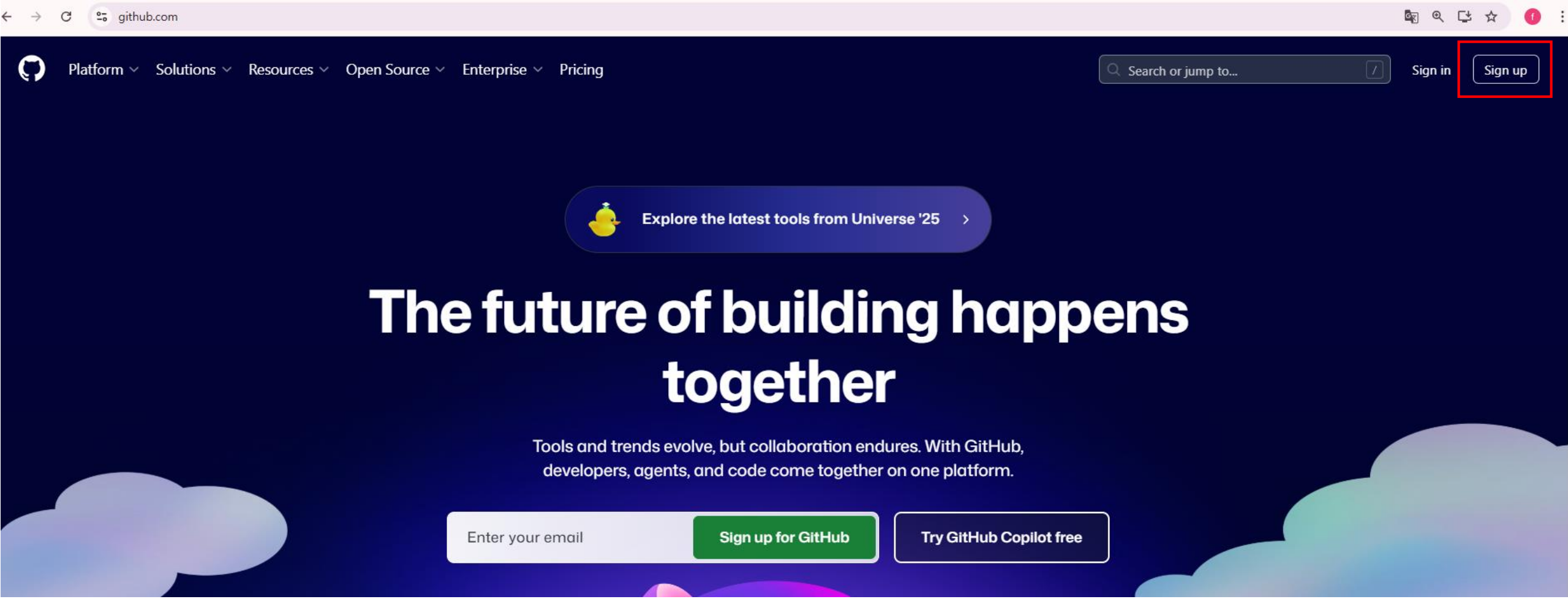
github를 이용한 코드 저장 및 버전 관리

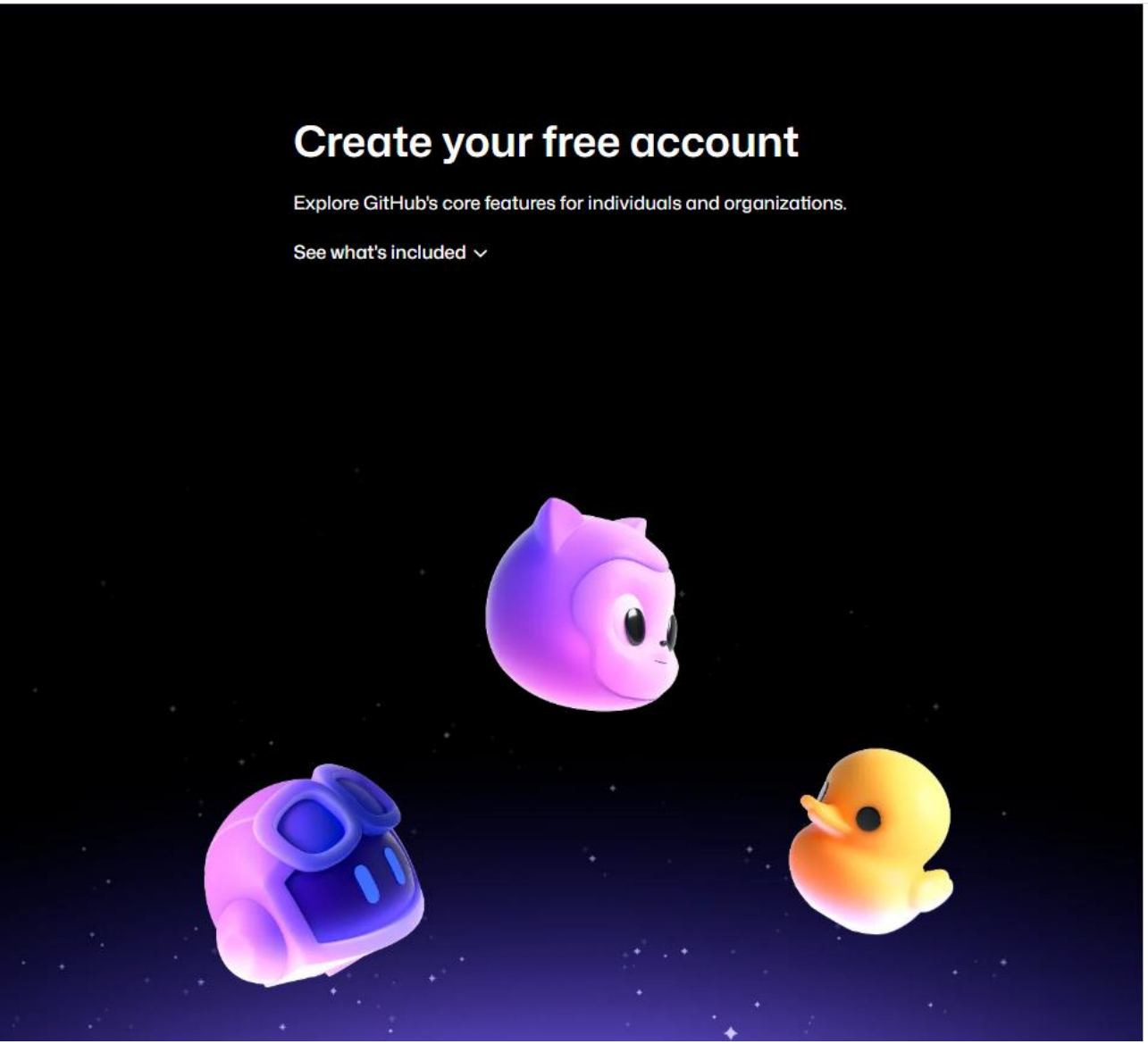
5) 로컬 저장소 생성: workspace 폴더 생성



github를 이용한 코드 저장 및 버전 관리

6) github.com 회원 가입, 새로운 저장소(repository) 생성





Already have an account? Sign in →

Sign up for GitHub

Continue with Google

Continue with Apple

or

Email*

Email

Password*

Password

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*

Username

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

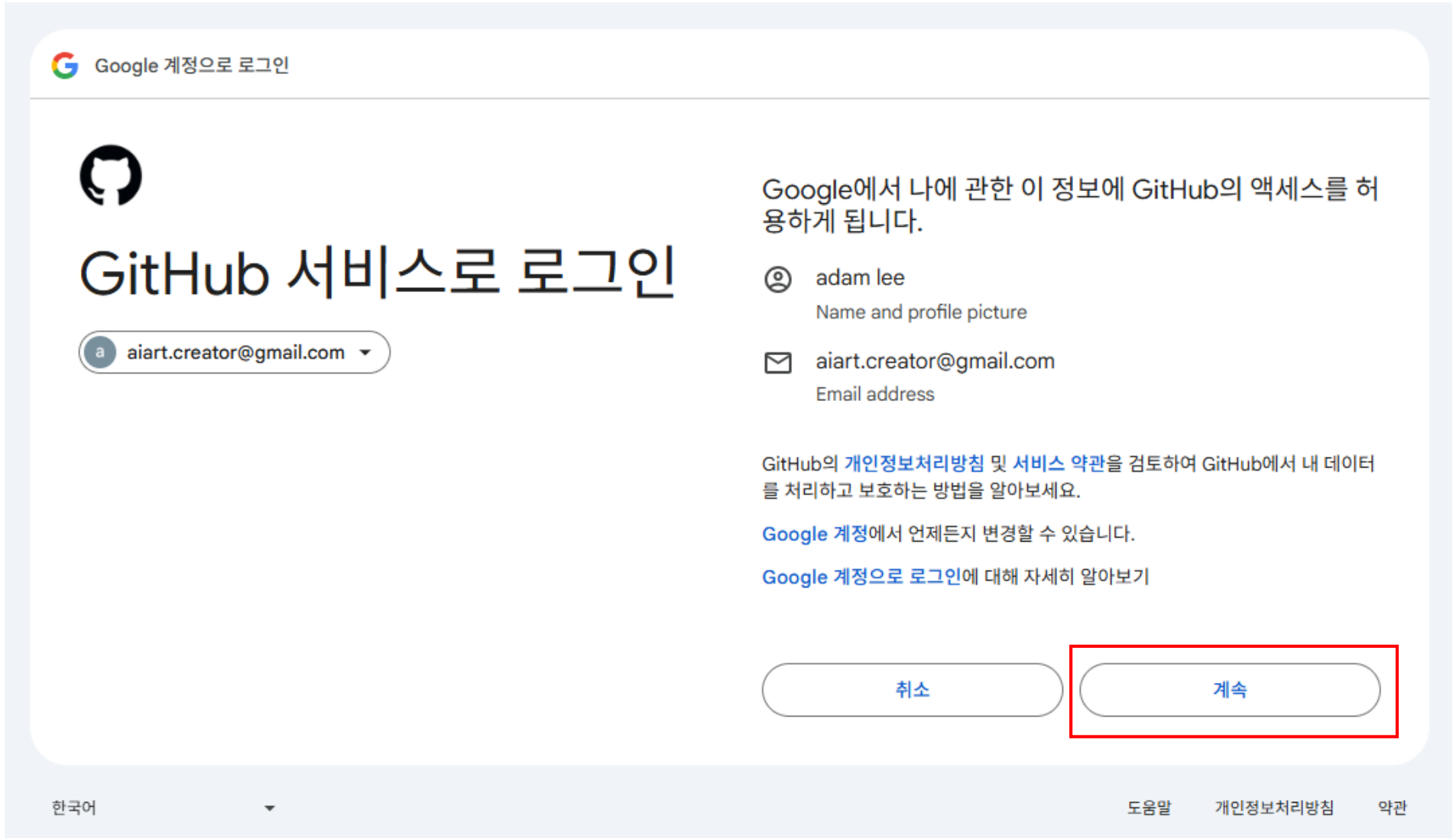
Your Country/Region*

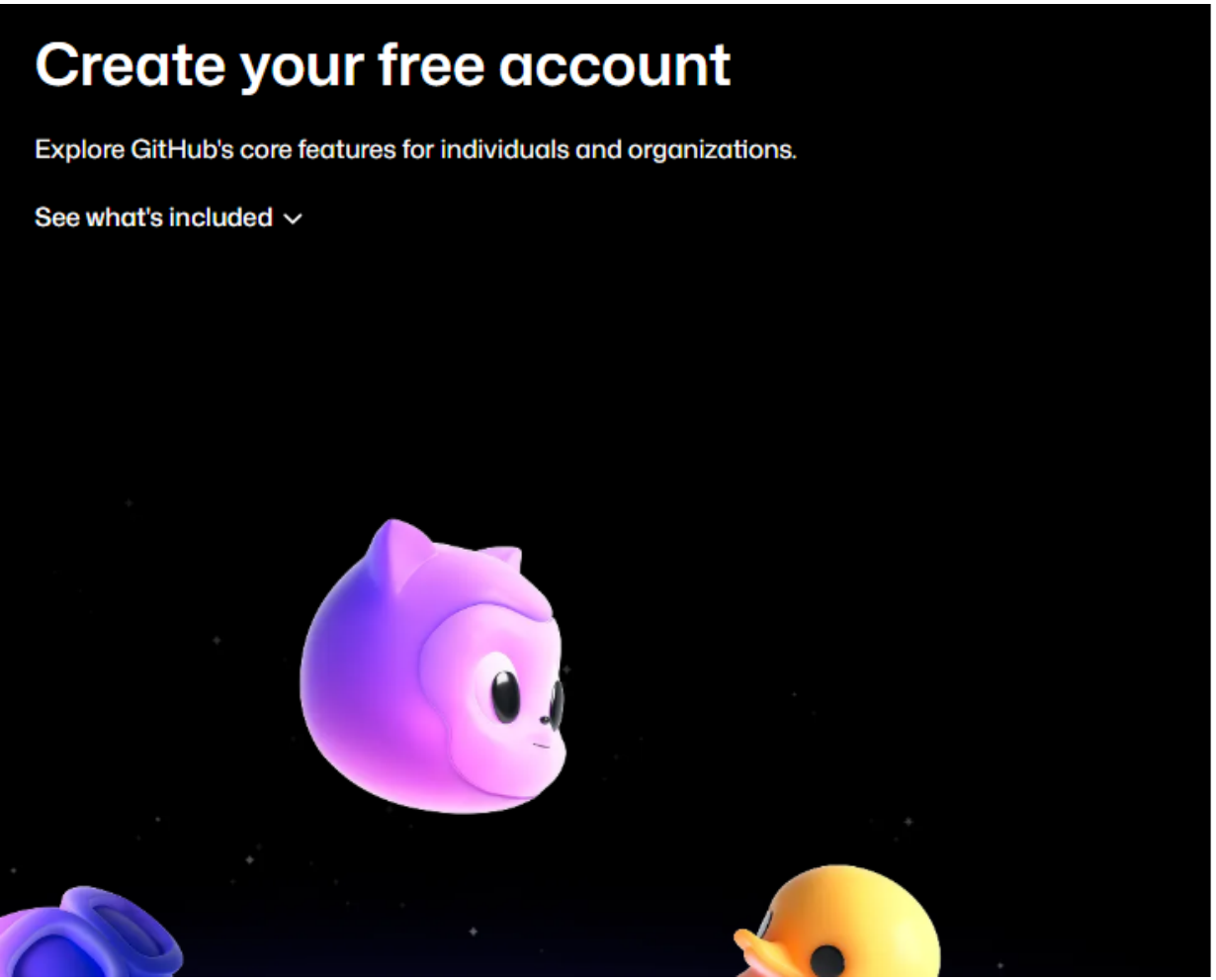
Korea, South

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Create account →

By creating an account, you agree to the Terms of Service. For more information


Google 계정으로 로그인
GitHub 서비스로 로그인
a aiart.creator@gmail.com
Google에서 나에 관한 이 정보에 GitHub의 액세스를 허용하게 됩니다.
adam lee
Name and profile picture
aiart.creator@gmail.com
Email address
GitHub의 개인정보처리방침 및 서비스 약관을 검토하여 GitHub에서 내 데이터를 처리하고 보호하는 방법을 알아보세요.
Google 계정에서 언제든지 변경할 수 있습니다.
Google 계정으로 로그인에 대해 자세히 알아보기
취소
계속
한국어
도움말
개인정보처리방침
약관



Sign up for GitHub

Email

aiart.creator@gmail.com

Use a different Google account

Username*

ai-educreator

Your Country/Region*

Korea, South

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Create account ›

By creating an account, you agree to the Terms of Service. For more information about GitHub's privacy practices, see the GitHub Privacy Statement. We'll occasionally send you account-related emails.

Create your first project

Ready to start building? Create a repository for a new idea or bring over an existing repository to keep contributing to it.

Create repository Import repository

Home

Try the new experience

Ask anything

+ Add repositories, files, and spaces

Task New Create issue Spark

Playlist

GitHub for beginners on YouTube

Designed to help you master the basics of GitHub, whether you're new to coding or looking to enhance your version control skills.

Start playlist



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

1 **General**

Owner *

ai-educreator /

Repository name *

git_test1

git_test1 is available.

Great repository names are short and memorable. How about legendary-barnacle?

Description

0 / 350 characters

2 **Configuration**

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. About READMEs

Off

Add .gitignore

.gitignore tells git which files not to track. About ignoring files

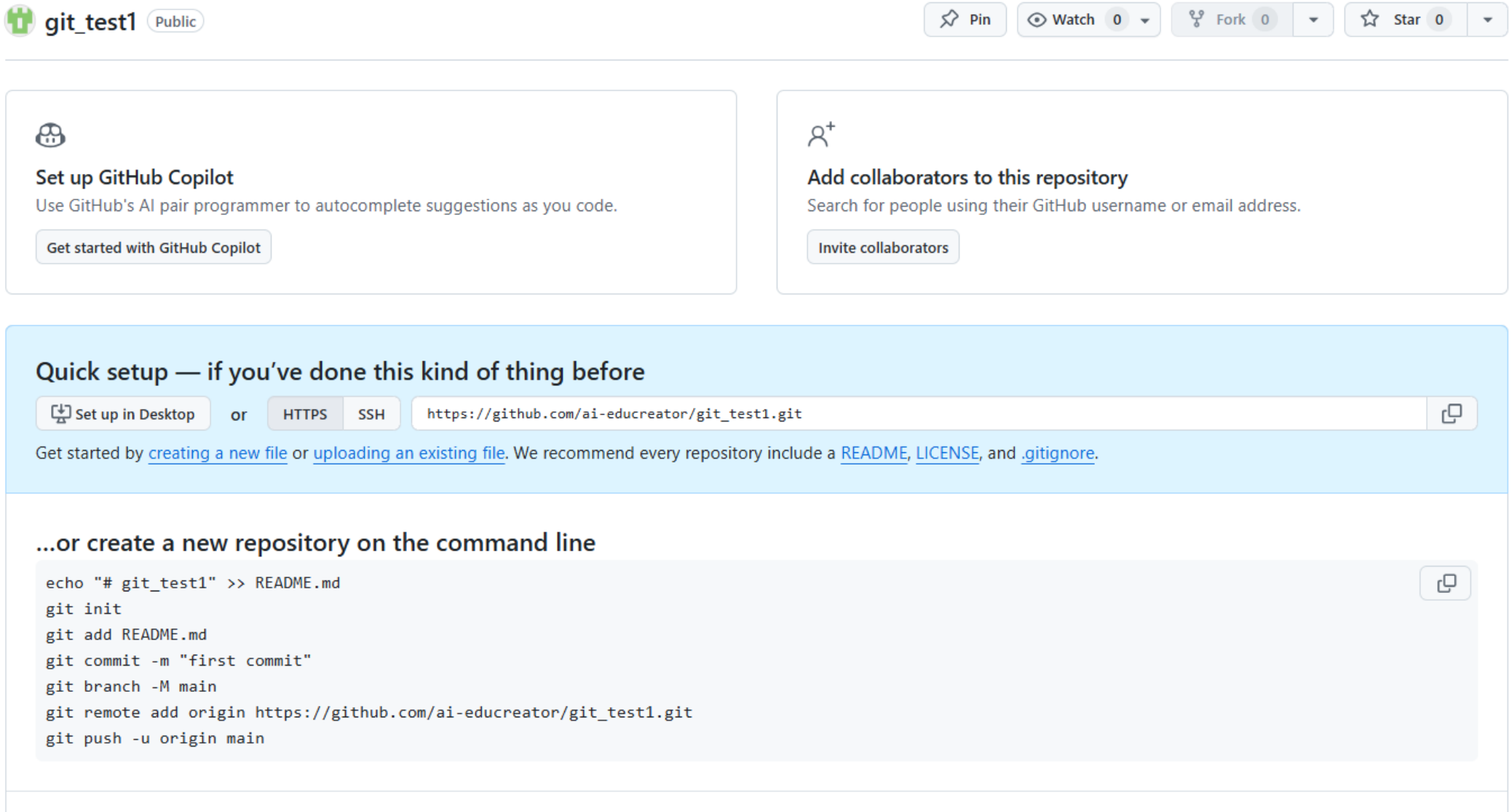
No .gitignore

Add license

Licenses explain how others can use your code. About licenses

No license

Create repository


git_test1
Public
Pin
Watch 0
Fork 0
Star 0
Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot
Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators
Quick setup — if you've done this kind of thing before
Set up in Desktop
or
HTTPS
SSH
https://github.com/ai-educreator/git_test1.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.
...or create a new repository on the command line
echo "# git_test1" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ai-educreator/git_test1.git
git push -u origin main

github를 이용한 코드 저장 및 버전 관리

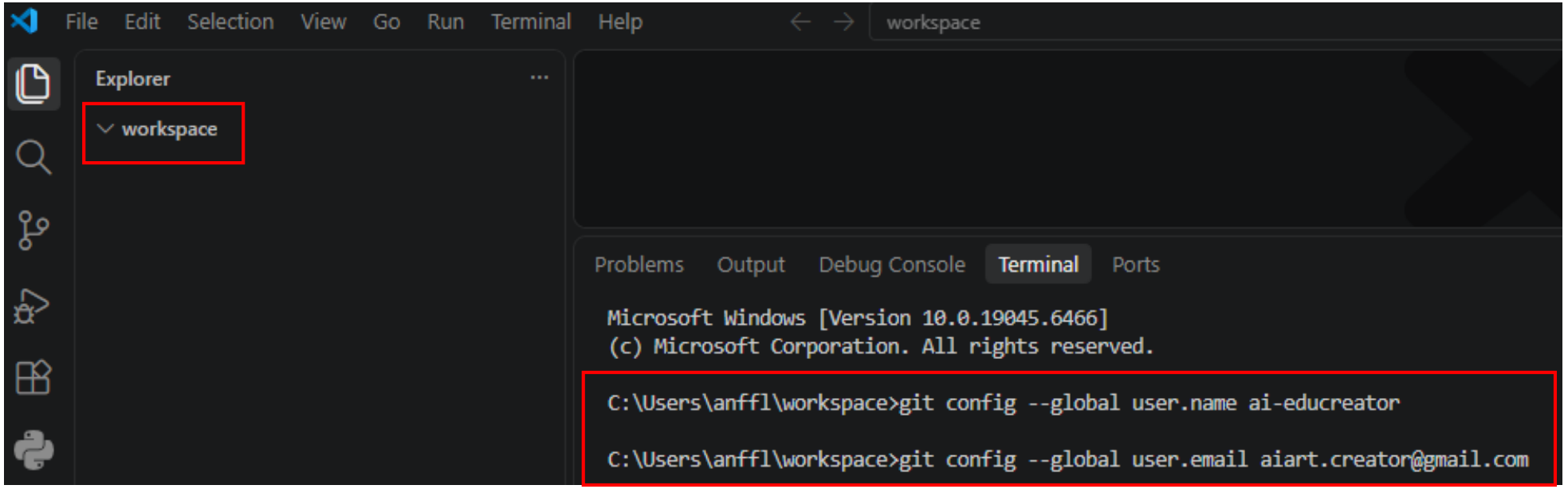
7) git 환경 설정

① 터미널 활성화

② 아래 명령어를 한 줄씩 입력하고 Enter 누름

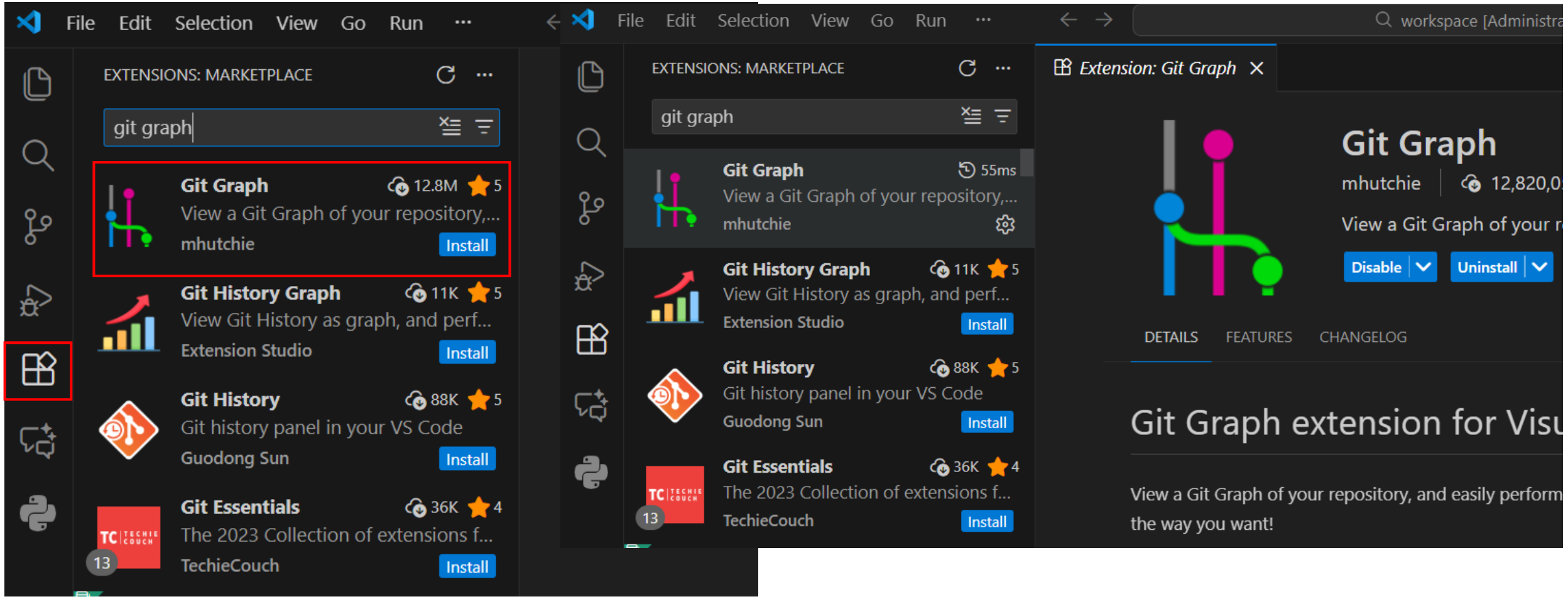
git config --global user.name "아무거나 사용: github.com 계정 이름"

git config --global user.email "github.com 계정 이메일"



github를 이용한 코드 저장 및 버전 관리

8) VS Code 확장 프로그램 설치: Git Graph(버전 변화 시각화)



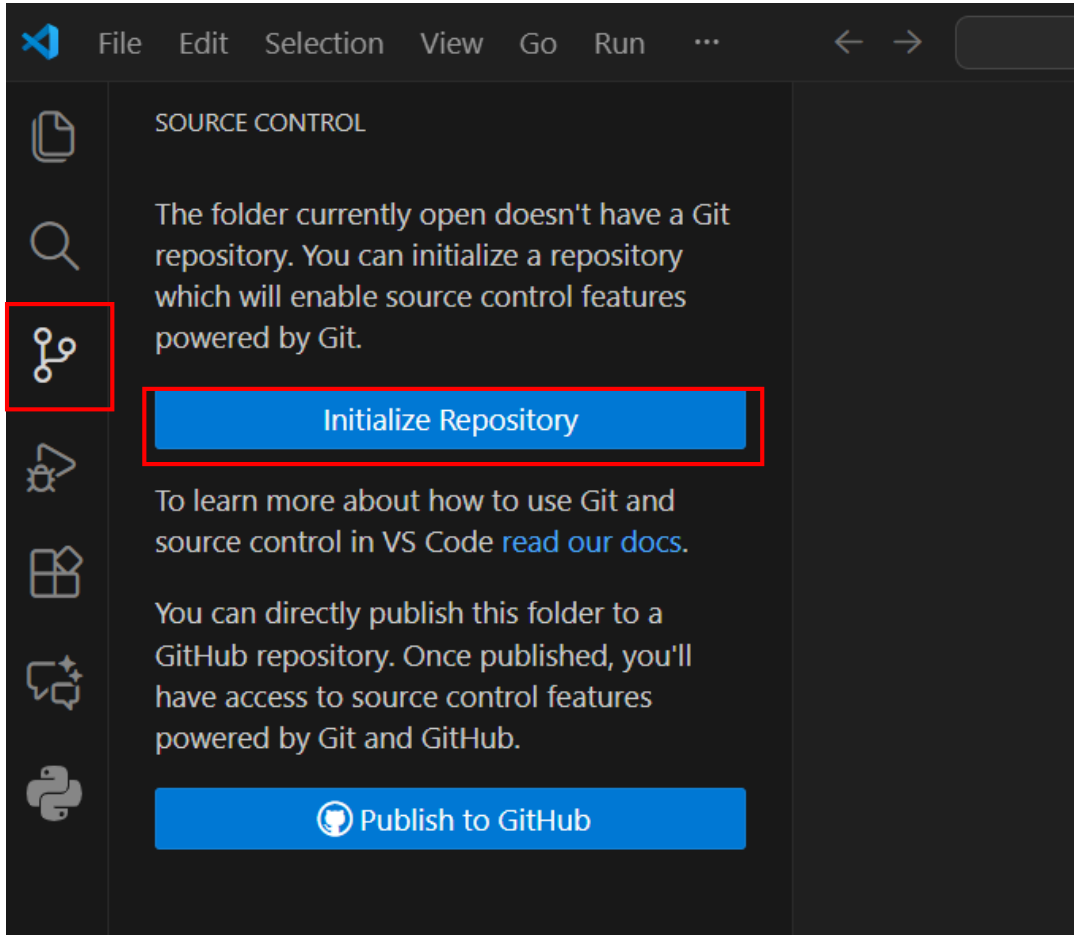
github를 이용한 코드 저장 및 버전 관리

9) 저장소 초기화(Initialize Repository)

① 작업 폴더 → 로컬 저장소 설정

② 작업 폴더에서 파일을 생성하고 코드 작성 및 저장

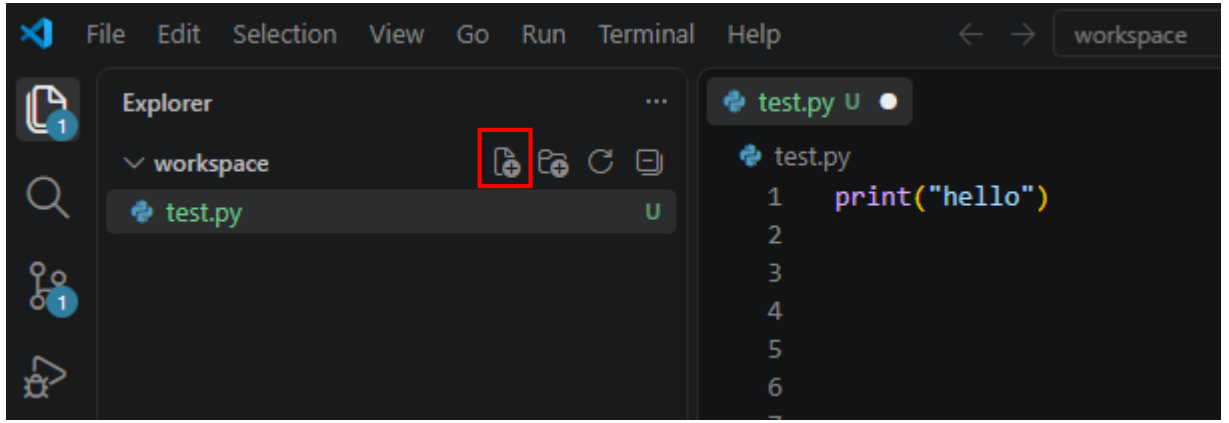
- Git이 이를 감지
- 작업 폴더 내부에 버전 관리용 .git 폴더(숨김 항목) 생성, 변화된 모든 사항 저장 → 버전 관리



github를 이용한 코드 저장 및 버전 관리

10) commit 실행

① 작업 폴더 → 파이썬 파일 생성

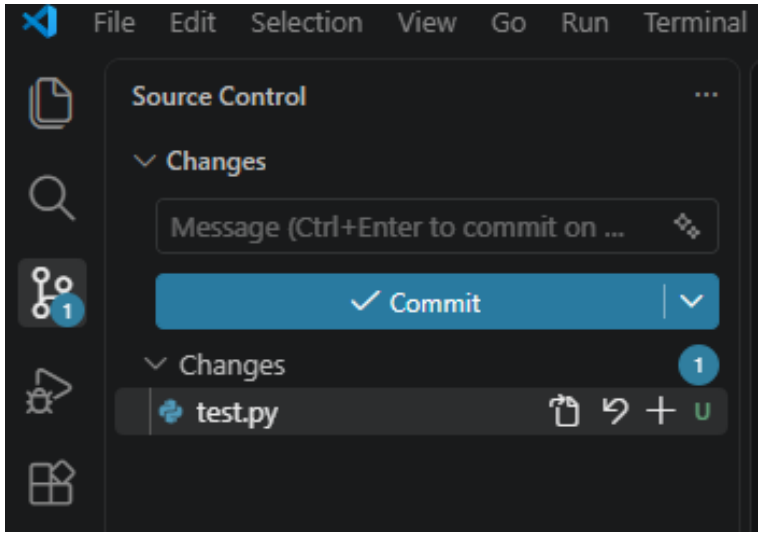


② U (Untracked): "Git이 이 파일을 감지했는데, 아직 commit(저장)하지 않았다"는 뜻

github를 이용한 코드 저장 및 버전 관리

11) commit 실행

③ 버전 관리의 3단계 흐름



- Source Control 아이콘 위에 파란색 숫자 ①이 떠 있음

  # test.py: 내가 작성한 파이썬 코드 파일

github를 이용한 코드 저장 및 버전 관리

11) commit 실행

③ 버전 관리의 3단계 흐름

- 숫자의 정체: "Git이 감지한 변화의 개수"

  # 의미: "방금 파일 1개가 새로 생기거나 변경되었다"

- [1단계] Changes (변경 사항) 확인

  # 현재 test.py가 있는 영역

  # 상태: 사용자가 commit(저장)하지 않은 상태

  # 특징: 파일 옆에 U (Untracked) 표시 "아직 한 번도 저장소에 저장한 적 없는 파일"이라는 뜻

github를 이용한 코드 저장 및 버전 관리

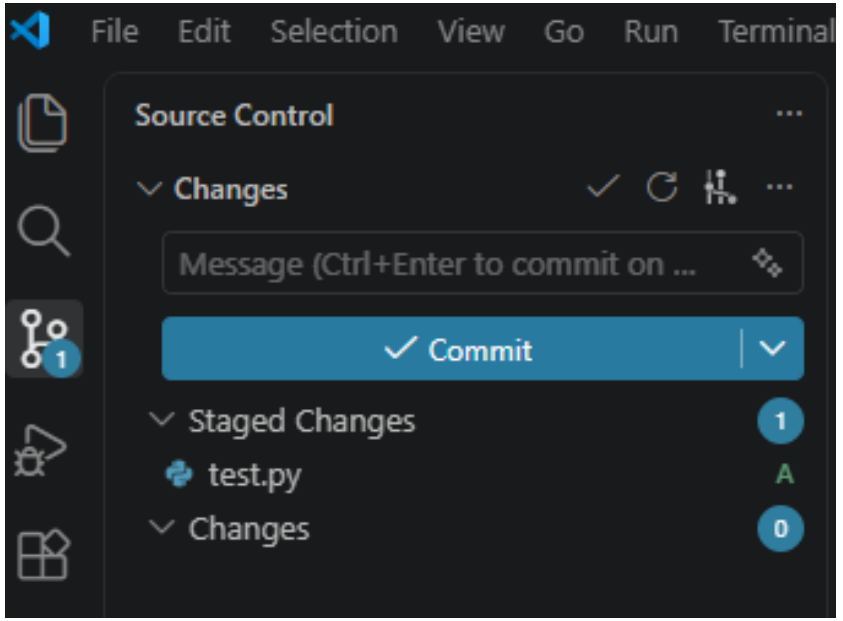
11) commit 실행

③ 버전 관리의 3단계 흐름

- [2단계] 선별 (Staging)

  # 상황: test.py 코드만 저장소에 첫 번째 버전으로 남기고 싶고, 다른 파일은 굳이 남기고 싶지 않다면?

  # test.py 파일 이름 오른쪽에 있는 + (Stage Changes) 버튼을 클릭 → [Staged Changes]로 이동



github를 이용한 코드 저장 및 버전 관리

11) commit 실행

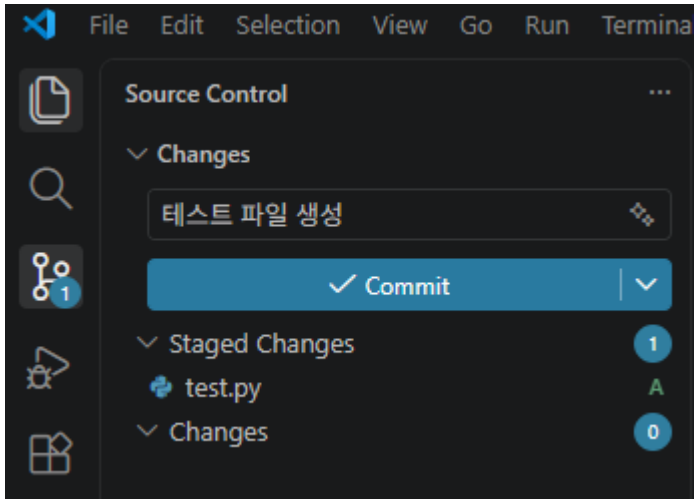
③ 버전 관리의 3단계 흐름

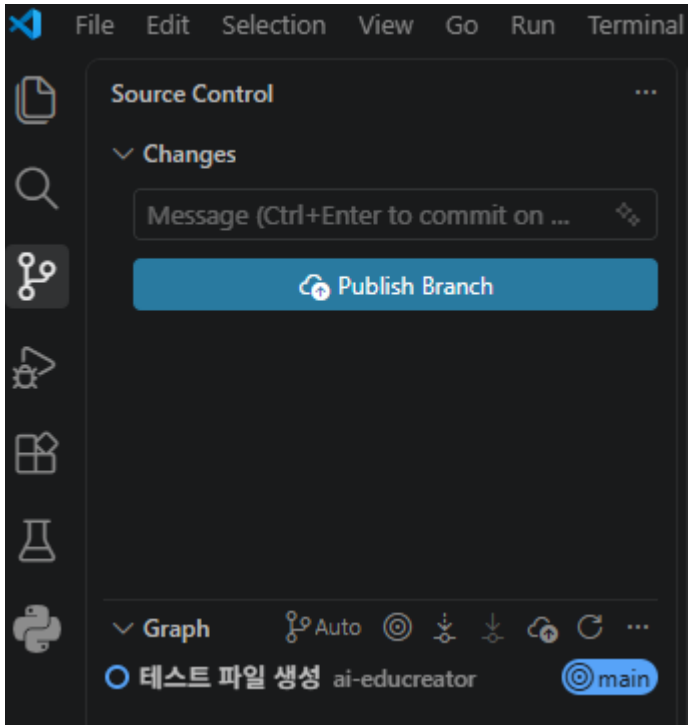
- [3단계] 확정 (Commit)

  # 메시지 입력창에 "테스트 파일 생성"라고 입력

  # Commit 버튼 클릭 → git 저장소로 영구 기록

  # 결과: test.py는 저장되었으므로 사라진다.

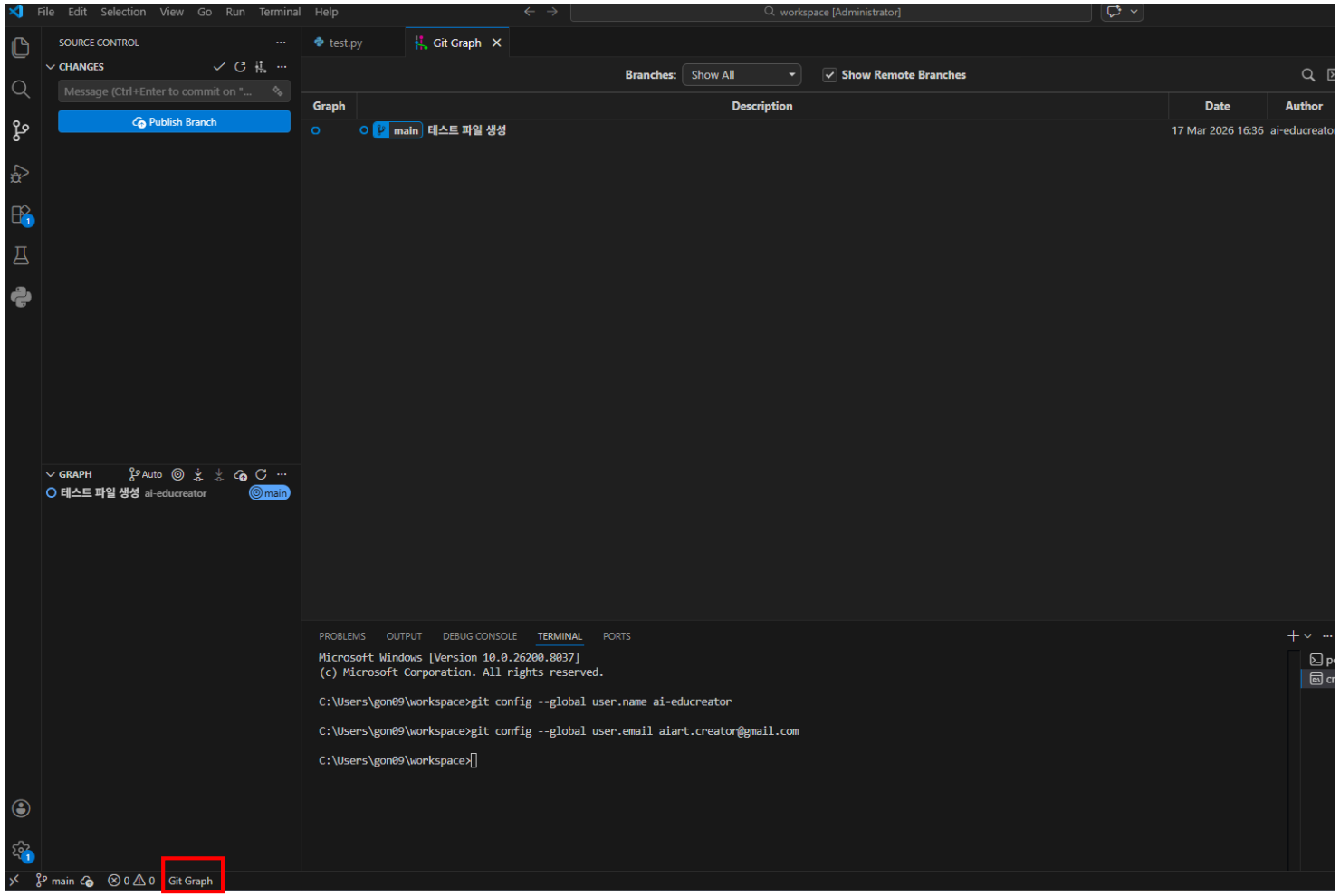




github를 이용한 코드 저장 및 버전 관리

11) commit 실행

④ Git Graph 아이콘 클릭



github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

① 로컬 저장소 (Local Repository): 내 컴퓨터 안의 .git 폴더. 나 혼자만의 일기장

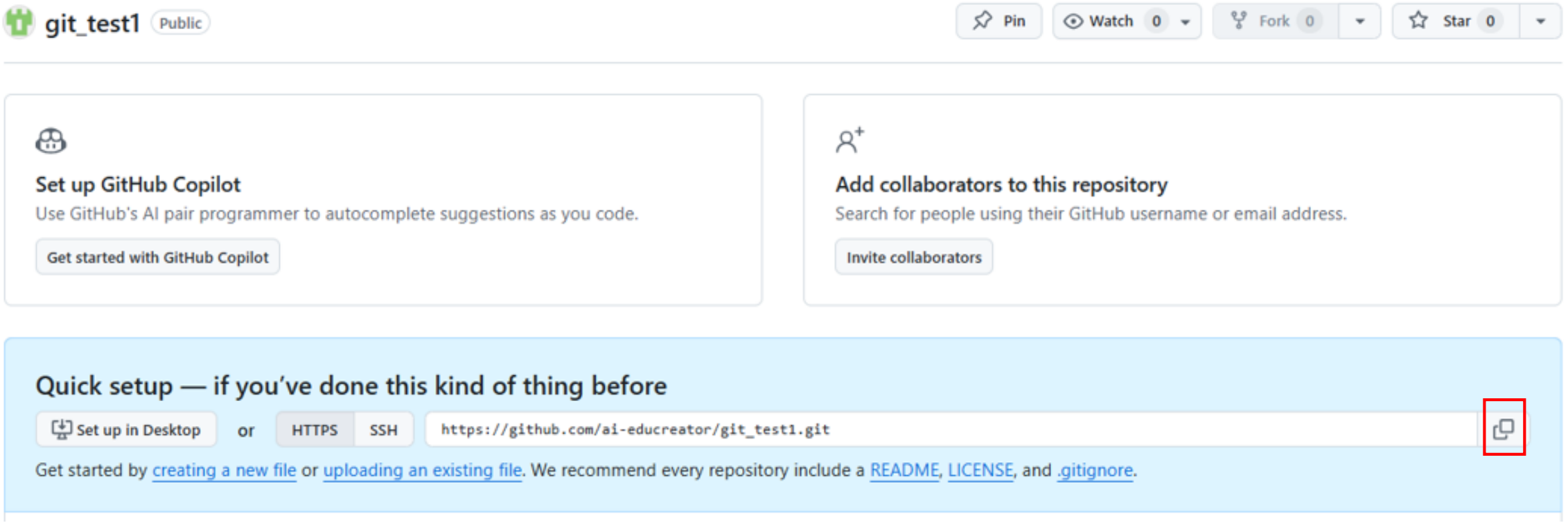
② 원격 저장소 (Remote Repository): GitHub 서버에 있는 저장소. 팀원과 공유하는 게시판

github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

③ 로컬 저장소와 원격 저장소 연결

• GitHub 레포지토리 주소 복사

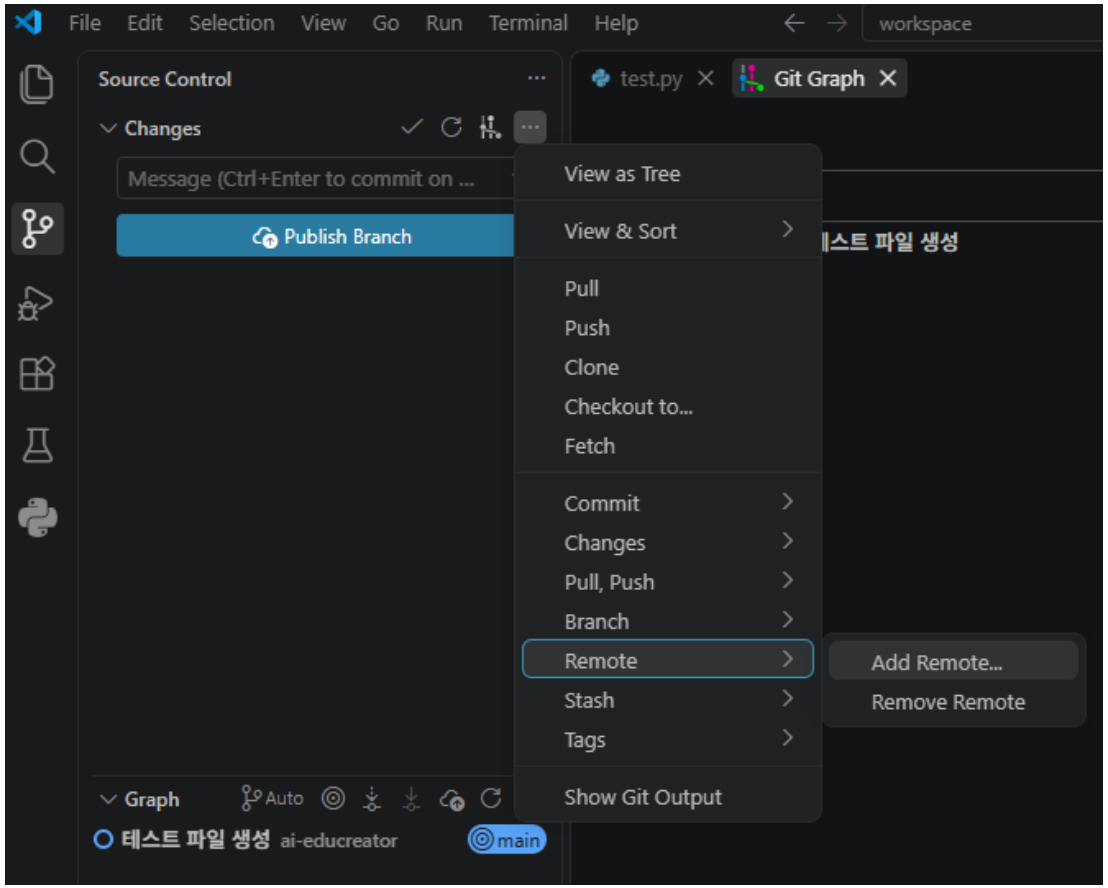


github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

③ 로컬 저장소와 원격 저장소 연결

- Source Control 탭 → CHAGES → ... 메뉴 클릭 → Remote → Add Remote

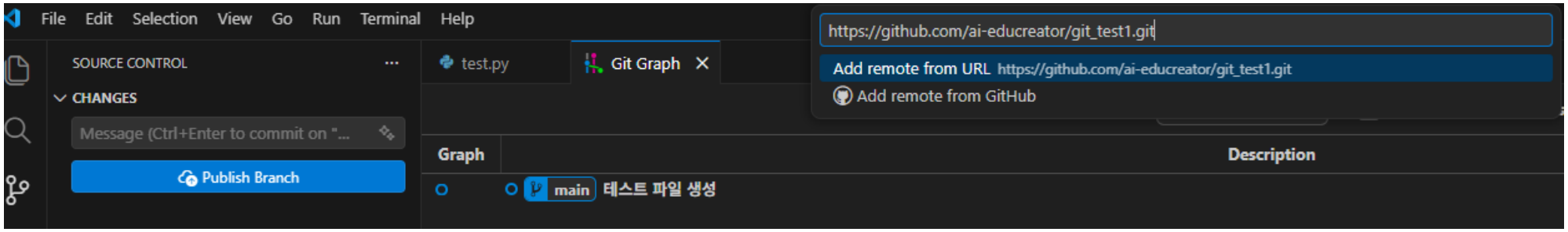


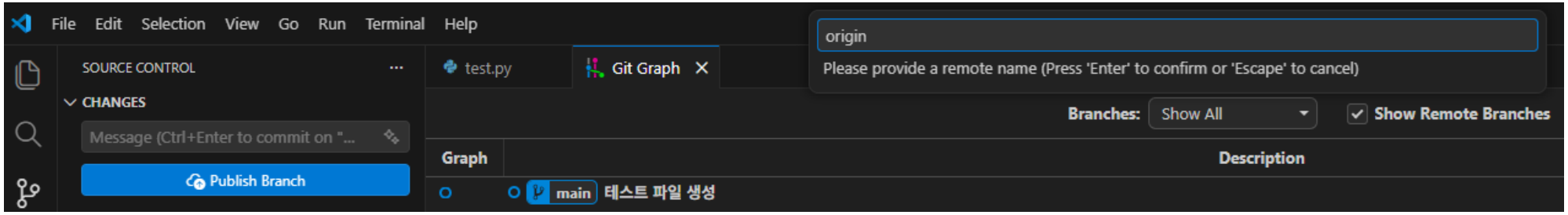
github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

③ 로컬 저장소와 원격 저장소 연결

- 복사한 주소 붙여놓고 Enter → origin 입력 (관례적으로 씀)하고 Enter



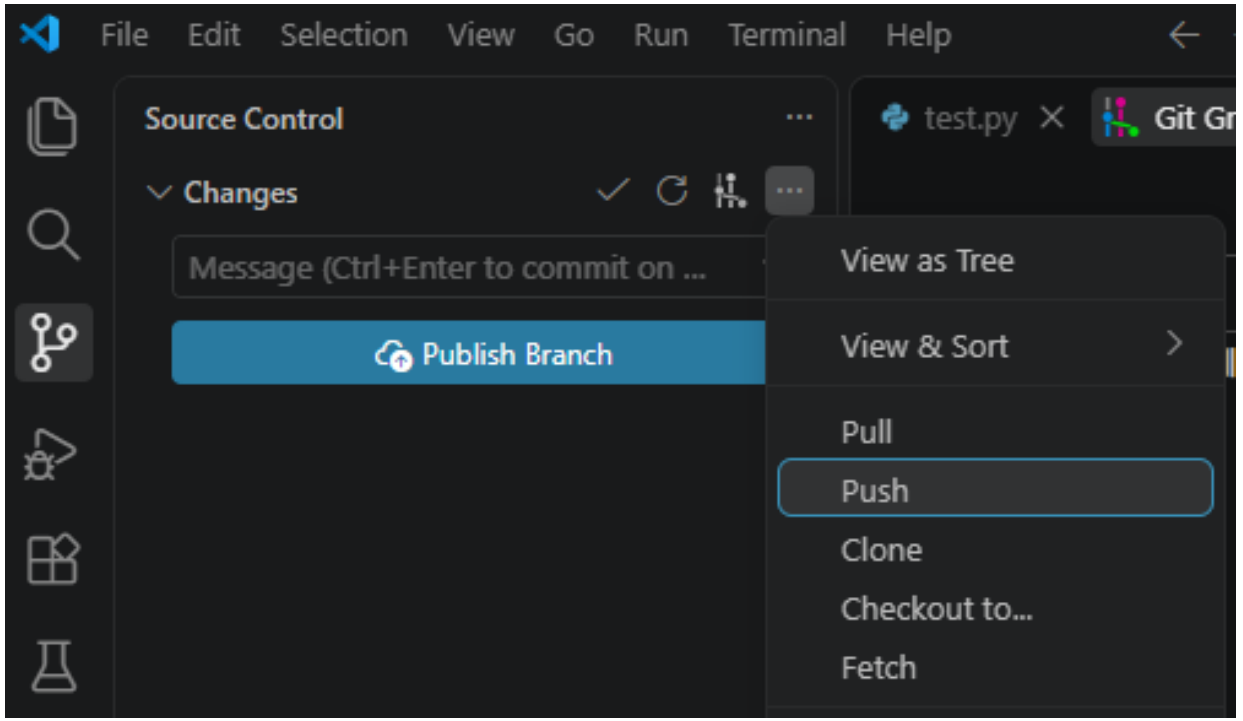


github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

④ Push : GitHub에 내 코드 올리기

• Push 버튼 클릭

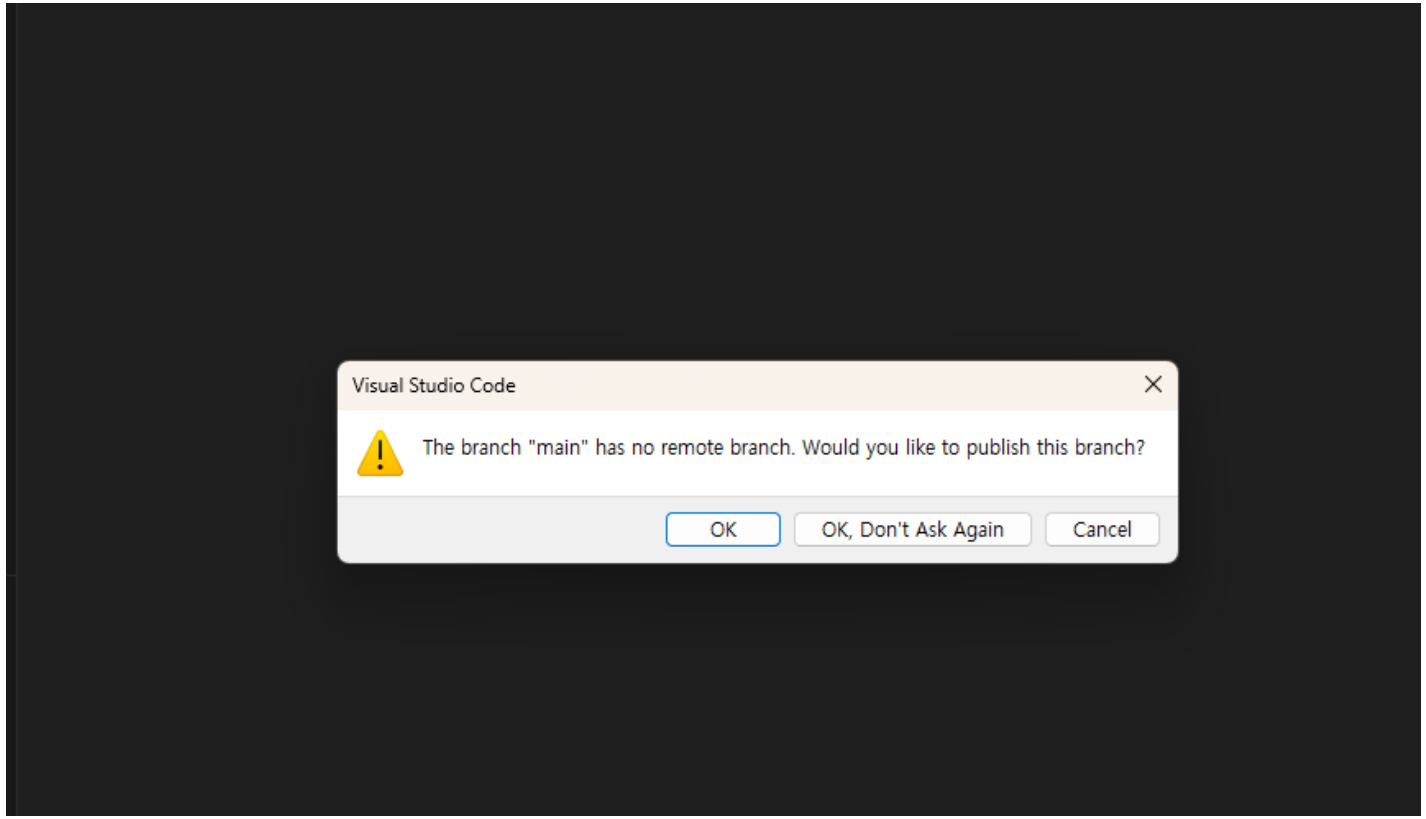


github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

④ Push : GitHub에 내 코드 올리기

• Push 버튼 클릭

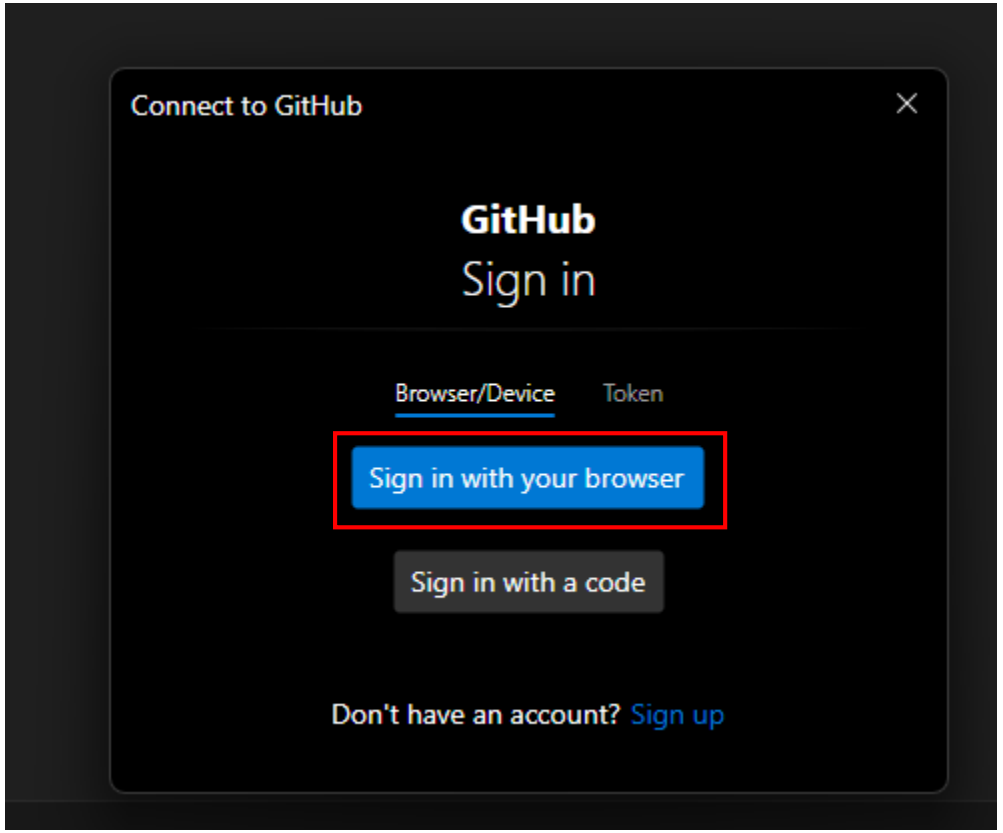


github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

④ Push : GitHub에 내 코드 올리기

• 인증 클릭

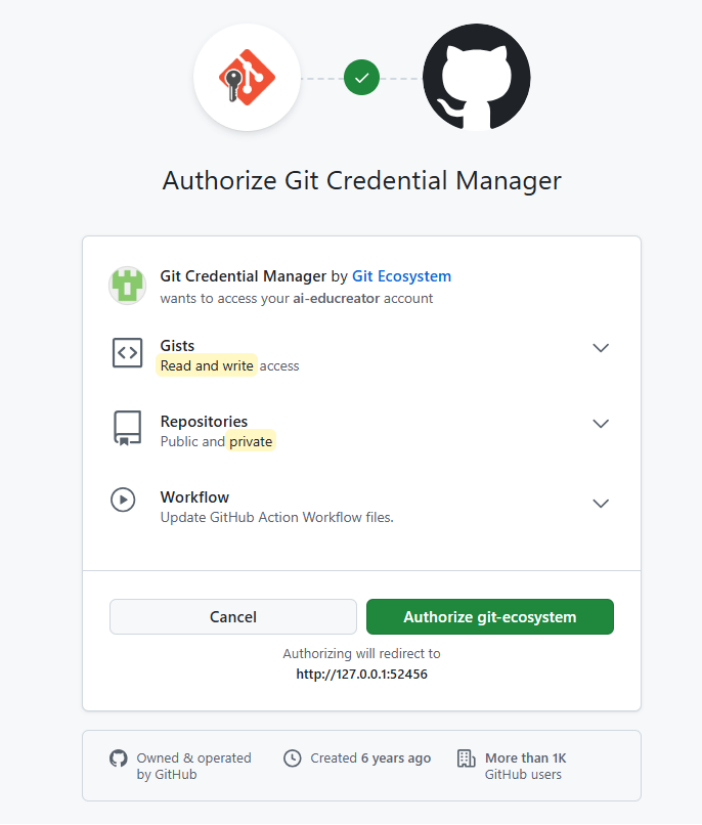


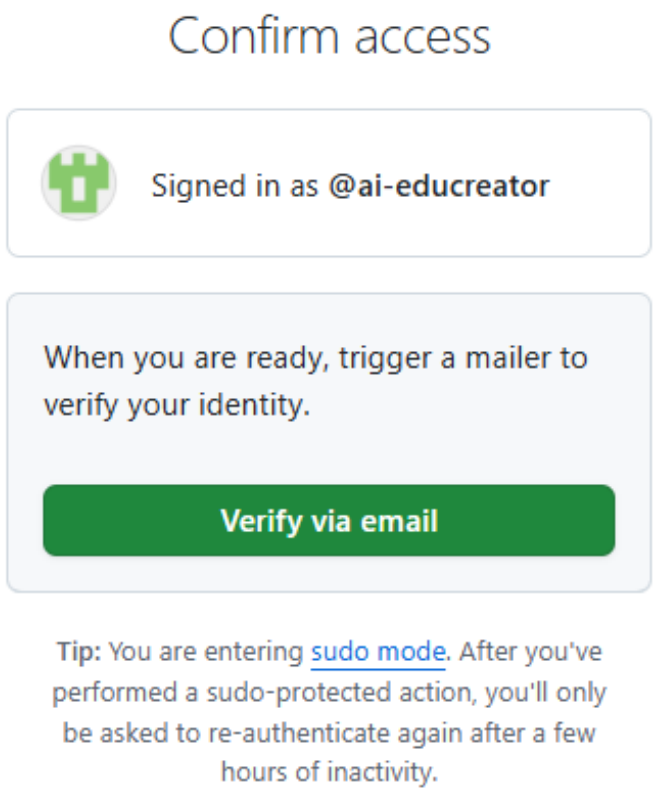
github를 이용한 코드 저장 및 버전 관리

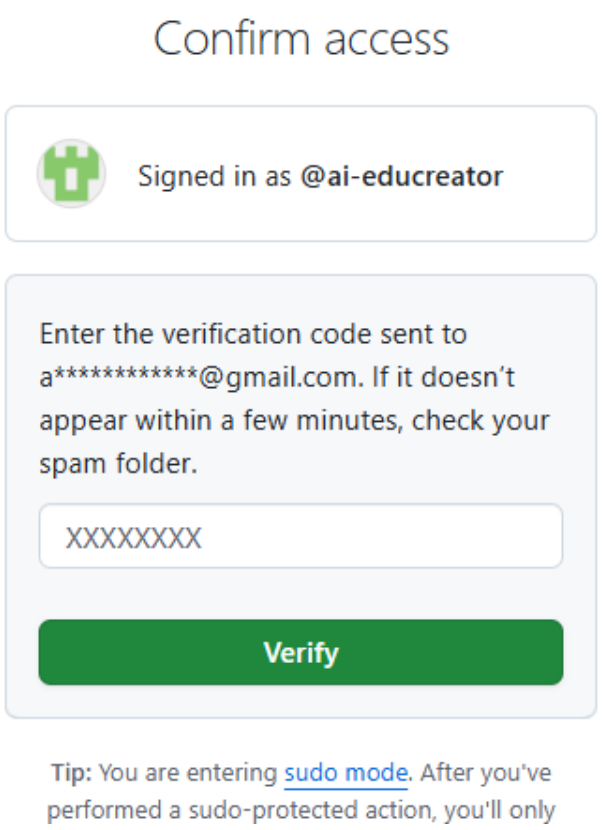
12) 저장소 업로드

④ Push : GitHub에 내 코드 올리기

• 인증 진행





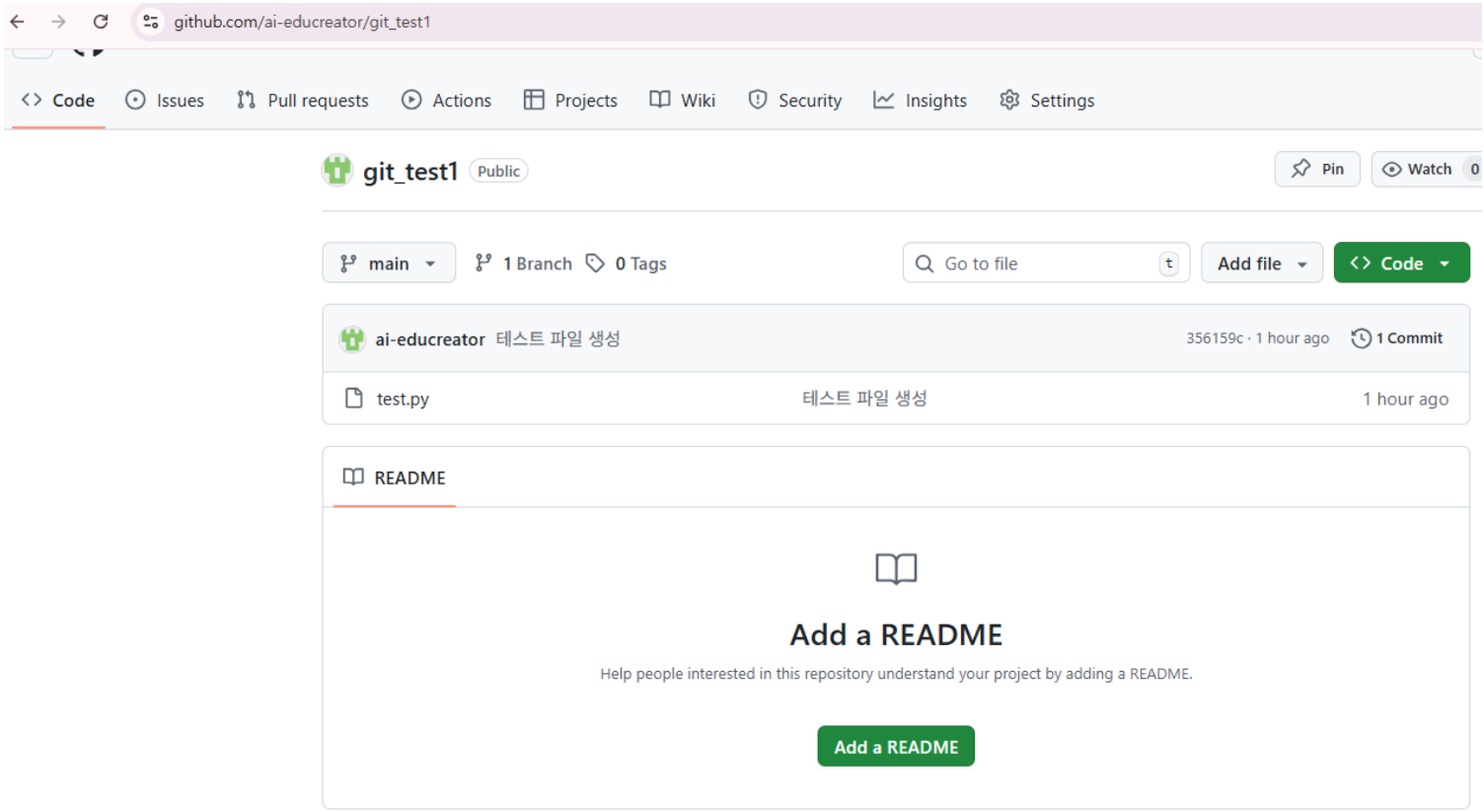


github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

④ Push : GitHub에 내 코드 올리기

- 완료: GitHub 웹페이지 새로고침 시 내 코드가 보이면 성공

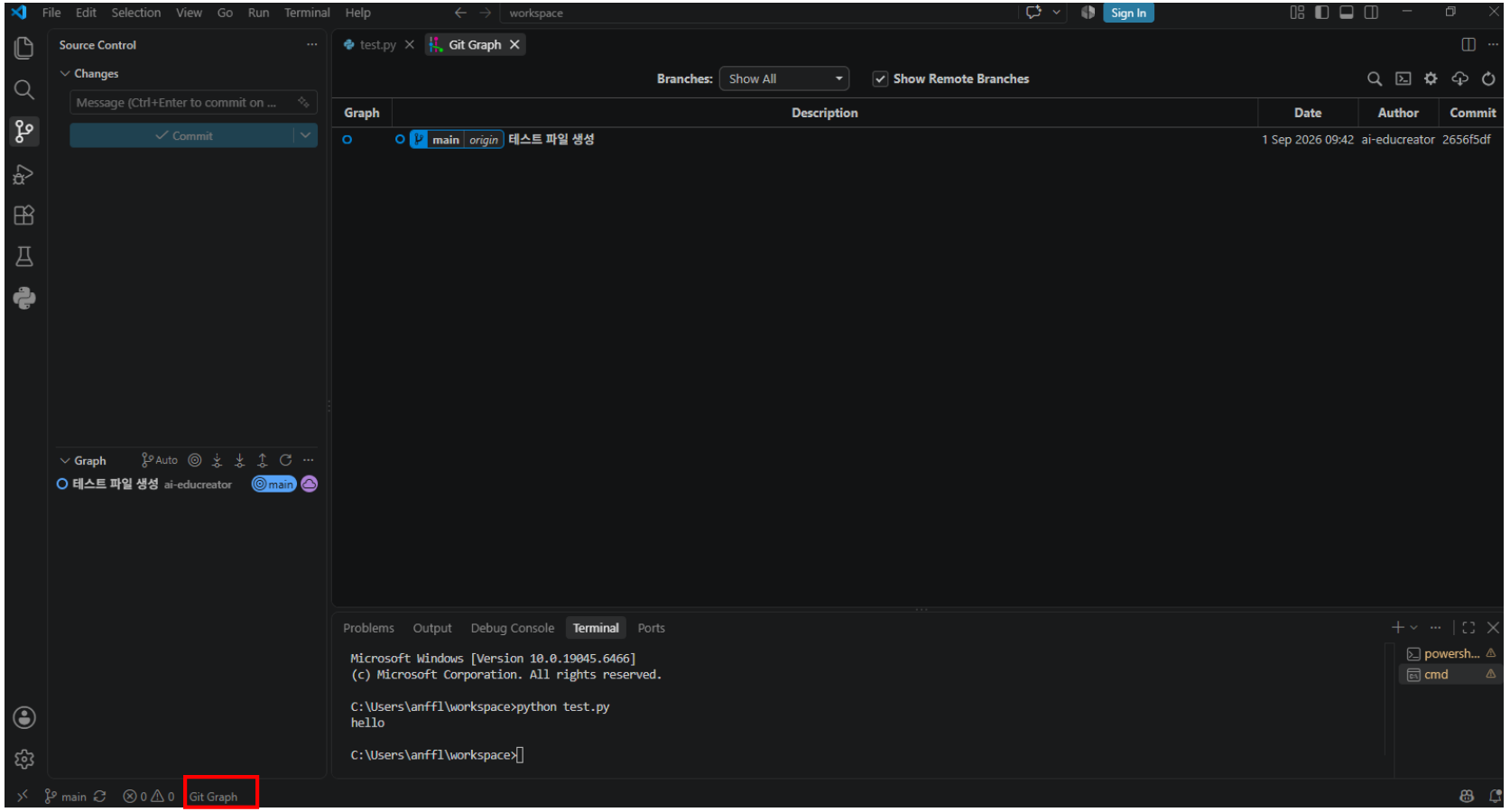


github를 이용한 코드 저장 및 버전 관리

12) 원격 저장소 업로드

④ Push : GitHub에 내 코드 올리기

- 완료



github를 이용한 코드 저장 및 버전 관리

13) Branch 활용하기

① Branch: 원본을 보호하는 '안전한 복사본'

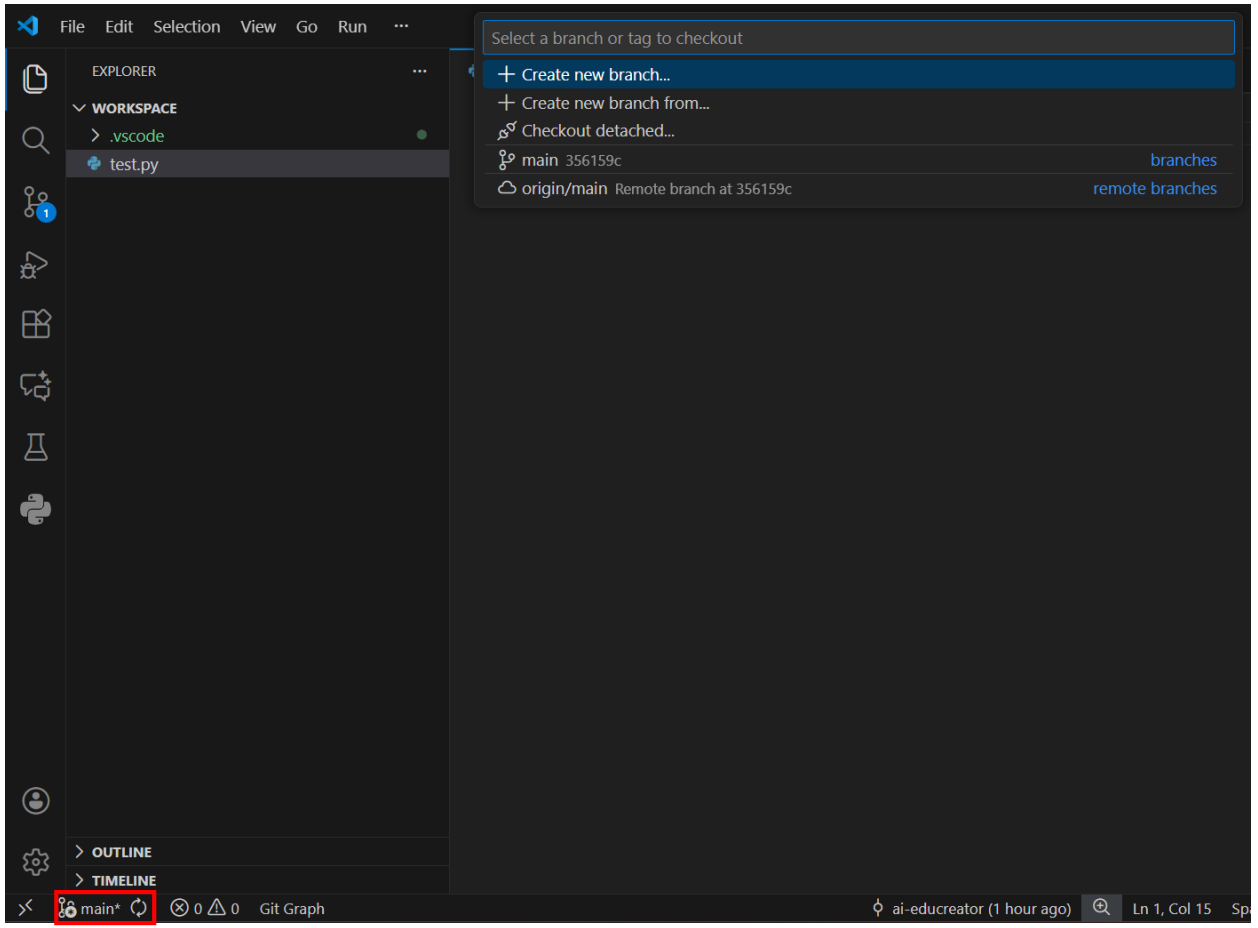
- 원본 코드(main)를 그대로 둔 채, 마음껏 테스트하고 버릴 수 있는 복사본을 만들 수 있음
- 테스트가 망하면 브랜치만 삭제하면 됨. 원본(main)은 안전함

github를 이용한 코드 저장 및 버전 관리

13) Branch 활용하기

② Branch 생성

- VS Code 좌측 하단 main 클릭 → Create new branch 클릭

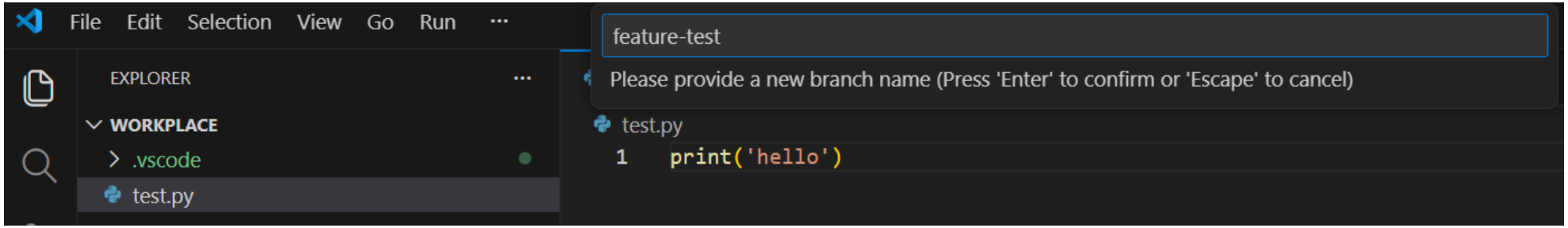


github를 이용한 코드 저장 및 버전 관리

13) Branch 활용하기

② Branch 생성

- new branch 이름 입력: feature-test 입력 후 Enter



github를 이용한 코드 저장 및 버전 관리

13) Branch 활용하기

② Branch 생성

- main → feature-test로 변경

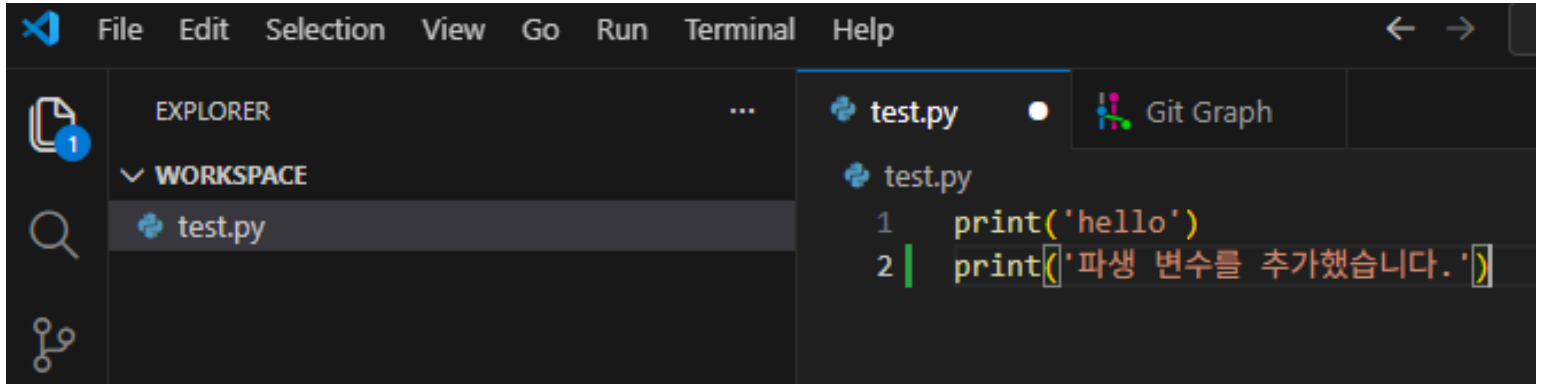


github를 이용한 코드 저장 및 버전 관리

13) Branch 활용하기

② Branch 생성

- test.py를 열고 코드 추가: print("파생 변수를 추가했습니다.")
- 저장

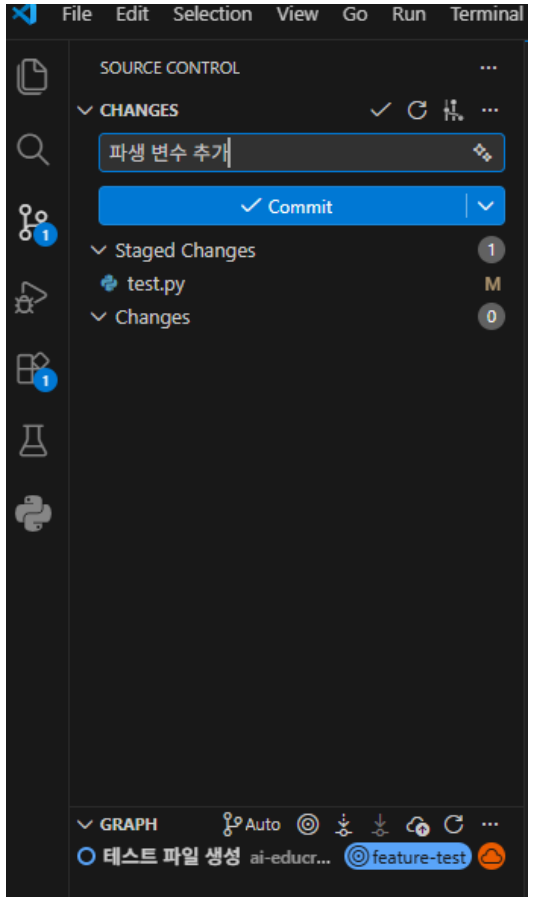


github를 이용한 코드 저장 및 버전 관리

13) Branch 활용하기

② Branch 생성

- 저장 → 선별 (Staging) → **Commit** (message: "파생 변수 추가")

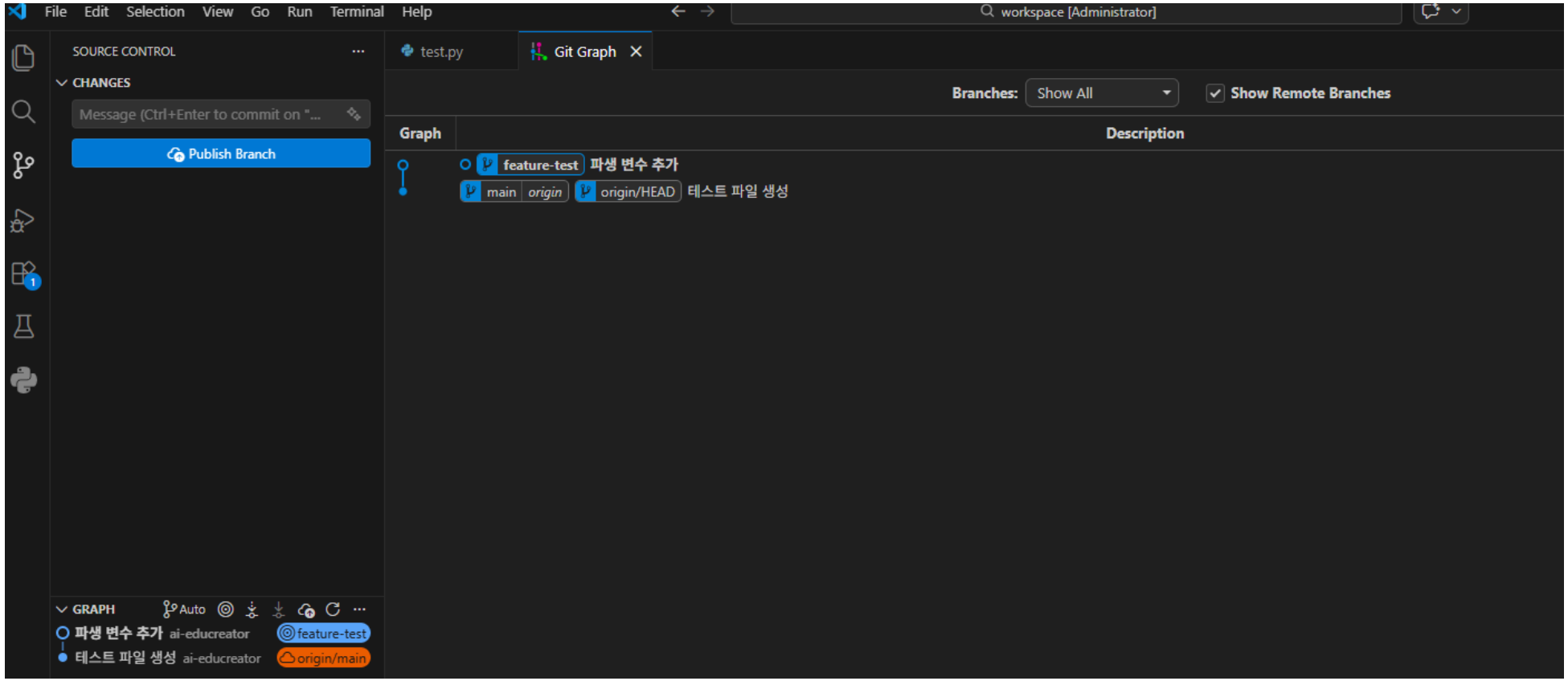


github를 이용한 코드 저장 및 버전 관리

13) Branch 활용하기

② Branch 생성

- 결과 확인: VS Code 좌측 하단 Git Graph 클릭

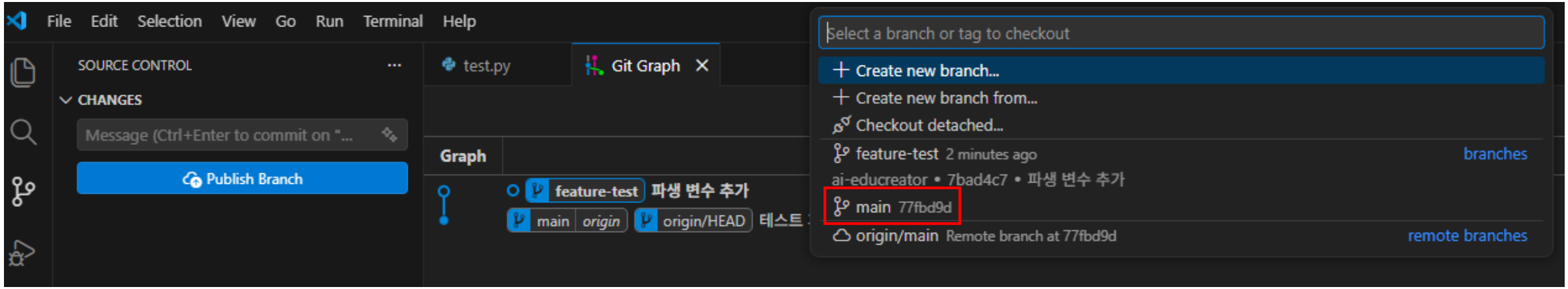


github를 이용한 코드 저장 및 버전 관리

13) Branch 활용하기

③ **복귀**: VS Code 좌측 하단 feature-test 클릭 → main 선택

- 관찰: 추가했던 print 코드가 감쪽같이 사라짐! ('원본'으로 돌아왔기 때문)
- 다시 feature-test로 돌아가면 추가했던 print 코드가 다시 나타남



github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

① GitHub Flow 전략이란?

- "내 작업물을 팀에게 제출하는 과정"
- "모든 작업은 새로운 가지(Branch)에서 시작하고, 검토 후에 합친다"는 단순한 원칙
- 기능: 팀 프로젝트 시 서로의 코드를 덮어쓰거나 망가뜨리지 않고, 안전하게 각자의 분석 작업을 진행한 뒤 검토를 거쳐 병합

② GitHub Flow 5단계

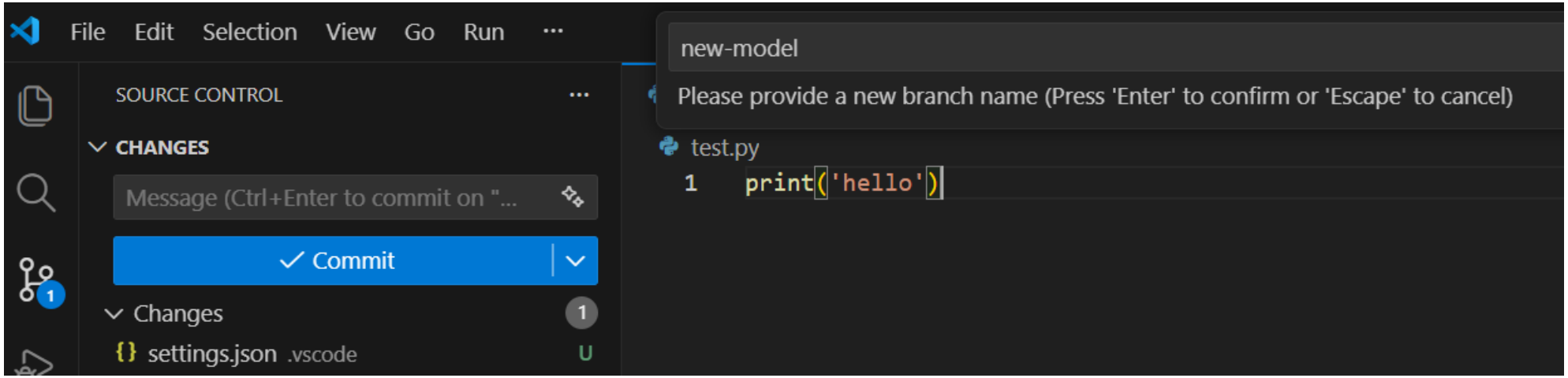
순서	장소	행동 (Action)	상세 설명 (예시)
Step 1	Local	브랜치 생성 (create)	(내 컴퓨터) main 파일은 건드리지 않고, new-model라는 Branch(복사본 파일) 생성
Step 2	Local	작업 및 커밋 (commit)	(내 컴퓨터) Branch(복사본 파일)에 코드를 짜고 저장
Step 3	Network	푸시 (push)	(전송) 내 컴퓨터에 있는 Branch(복사본 파일)을 GitHub 웹사이트로 업로드(Push)
Step 4	GitHub	풀 리퀘스트 (PR)	(GitHub 웹) "팀장님, 제가 올린 new-model 확인해 주시고, 문제없으면 main에 합쳐주세요!"라고 결재 요청
Step 5	GitHub	병합 (Merge)	(GitHub 웹) 팀장이 "승인" 버튼을 누르면, GitHub 웹사이트 상의 main 코드가 업데이트

github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 1단계: VS Code 좌측 하단에서 main을 클릭해 new-model 브랜치 생성

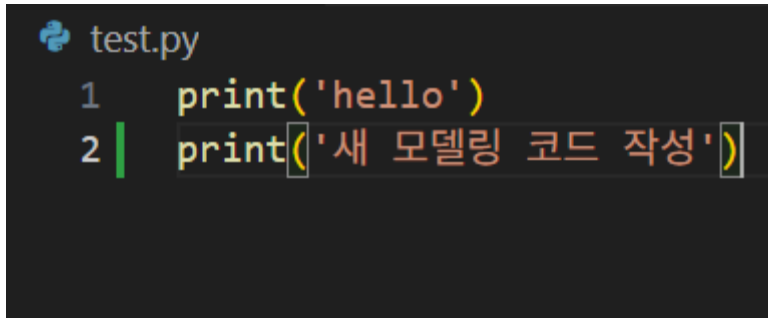


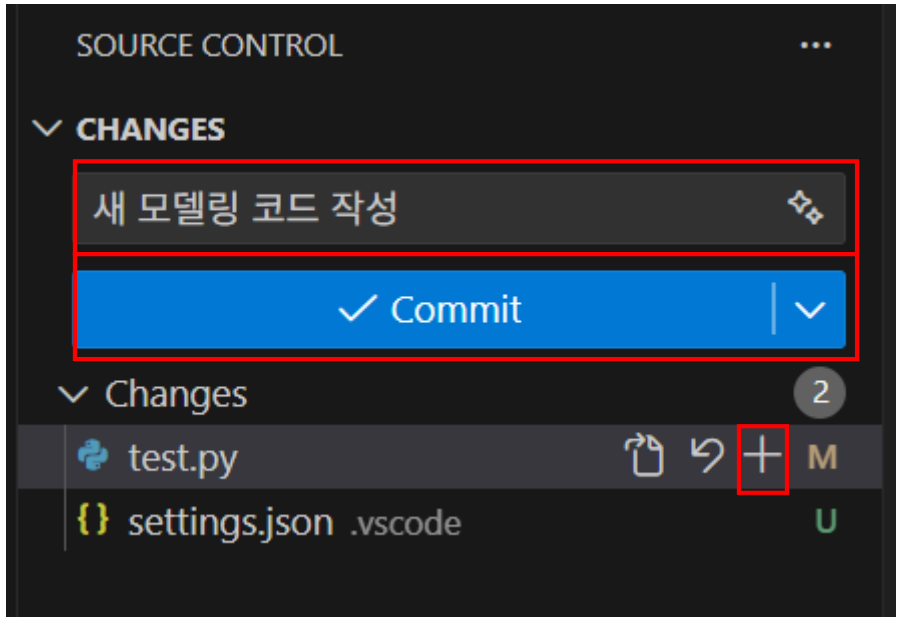
github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 2단계: 코드 수정 및 commit
  - message: 새 모델링 코드 작성



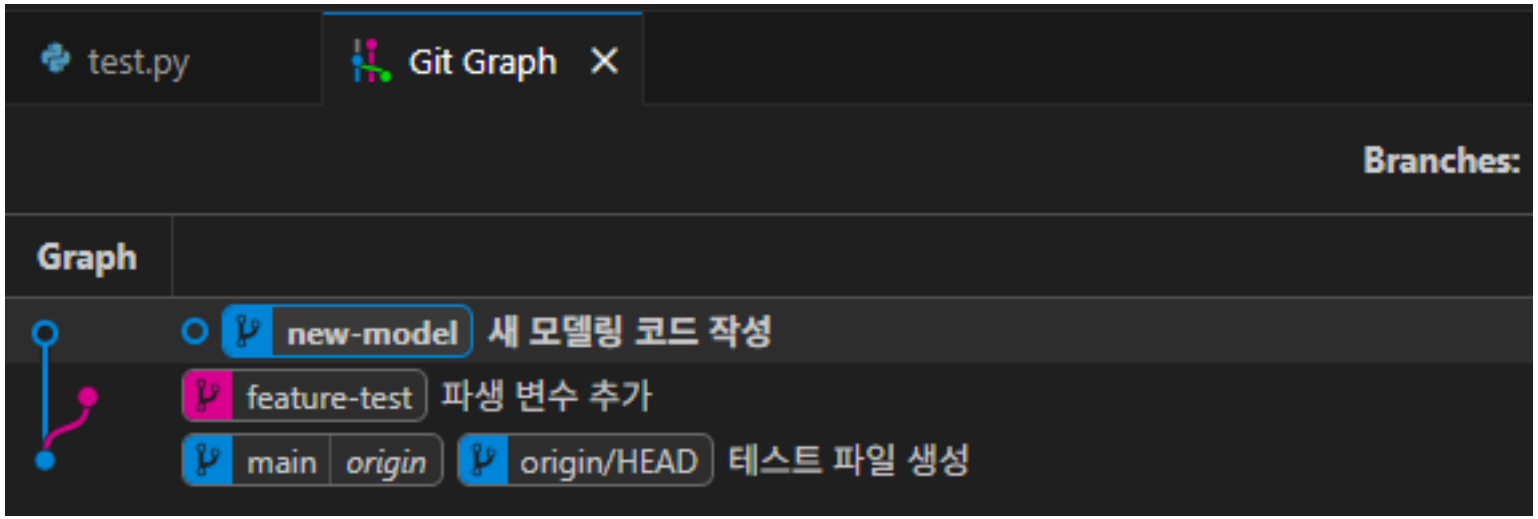


github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 2단계: 코드 수정 및 commit
  - message: 새 모델링 코드 작성

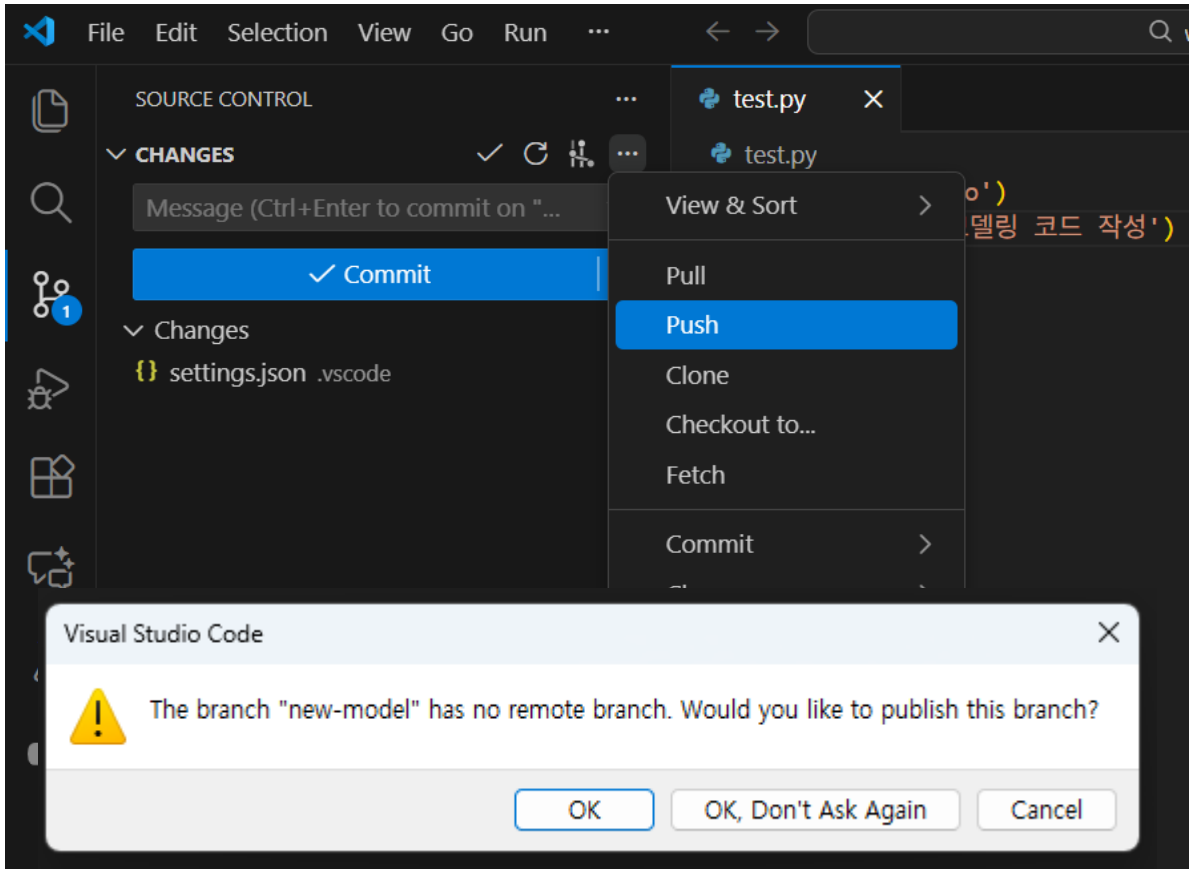


github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 3단계: VS Code에서 Push 버튼 클릭

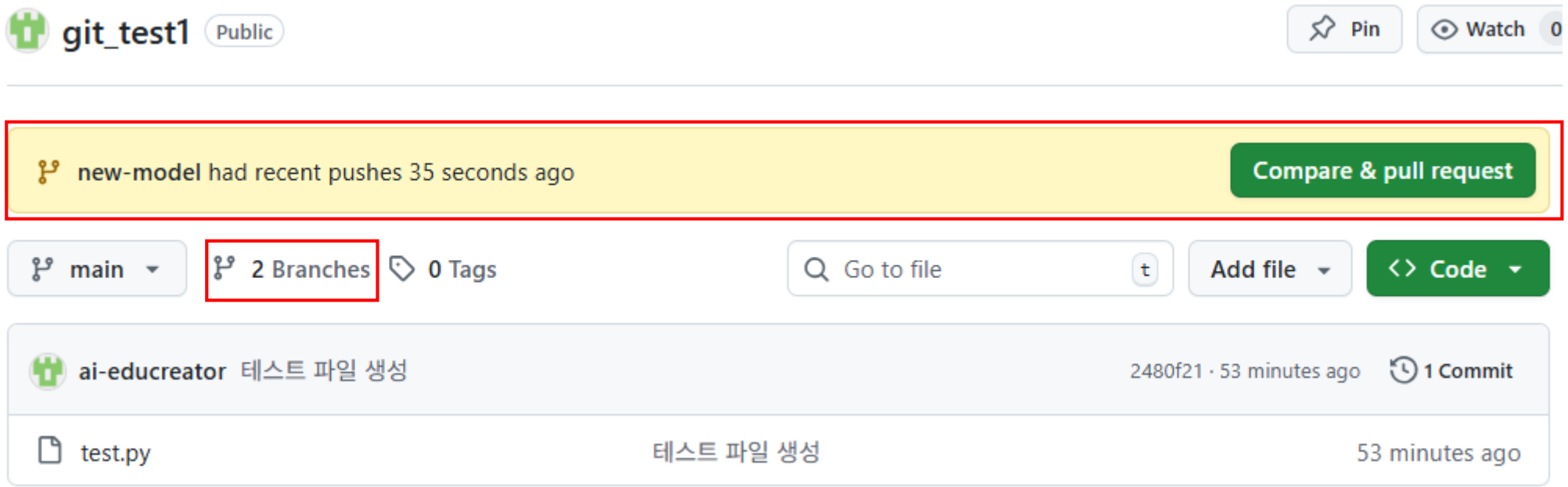


github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

• 4단계: "Compare & pull request" 버튼 생성 → 클릭

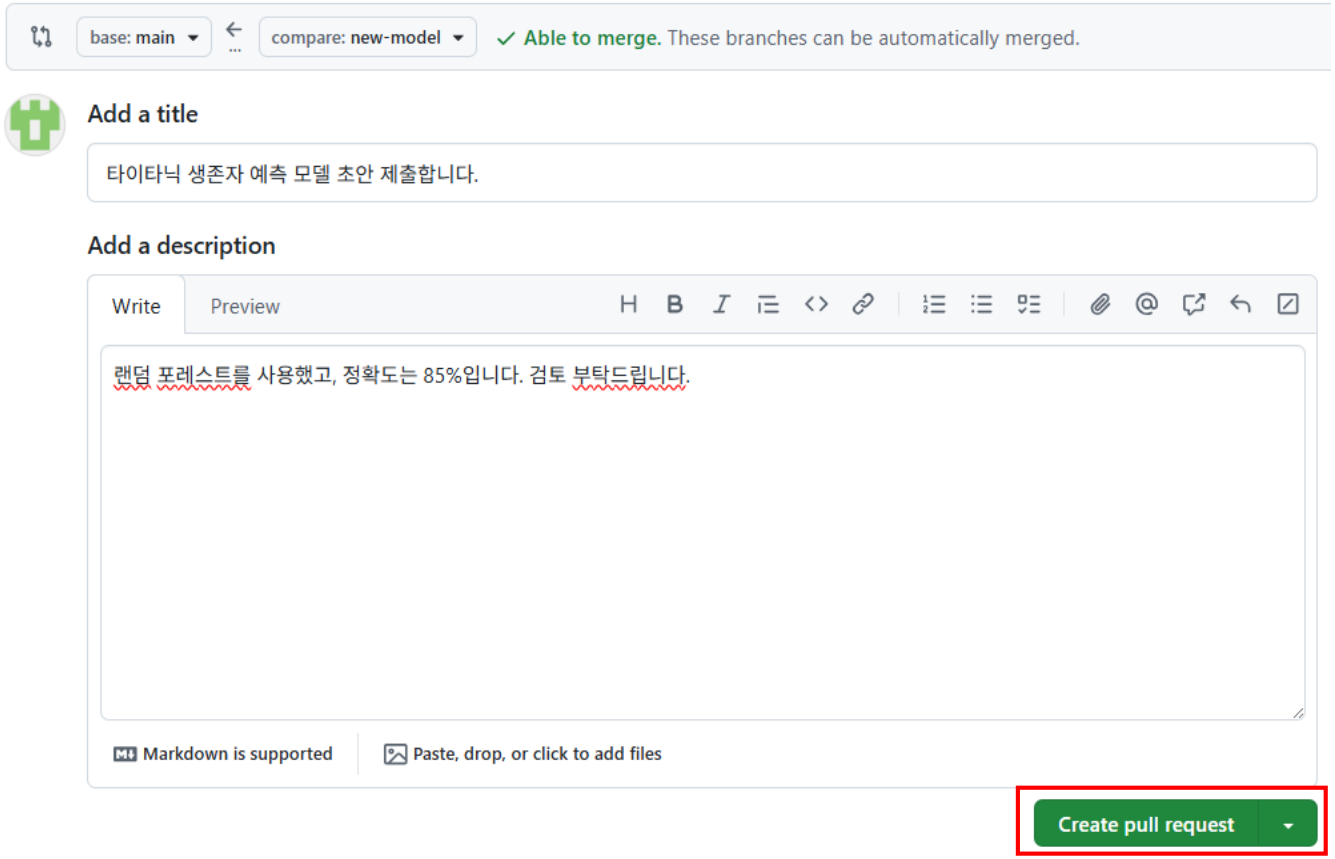


github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 4단계: 결재 신청서 작성 → create pull request 버튼 클릭

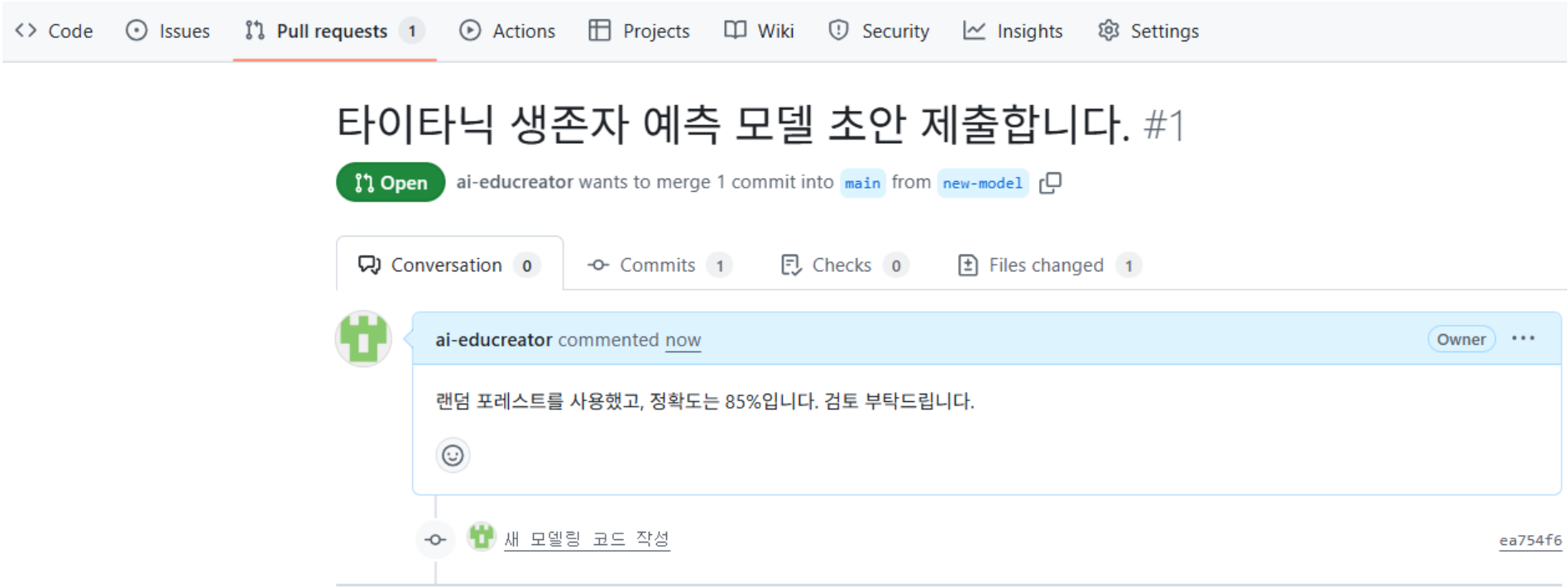


github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 4단계: Pull requests 탭 활성화

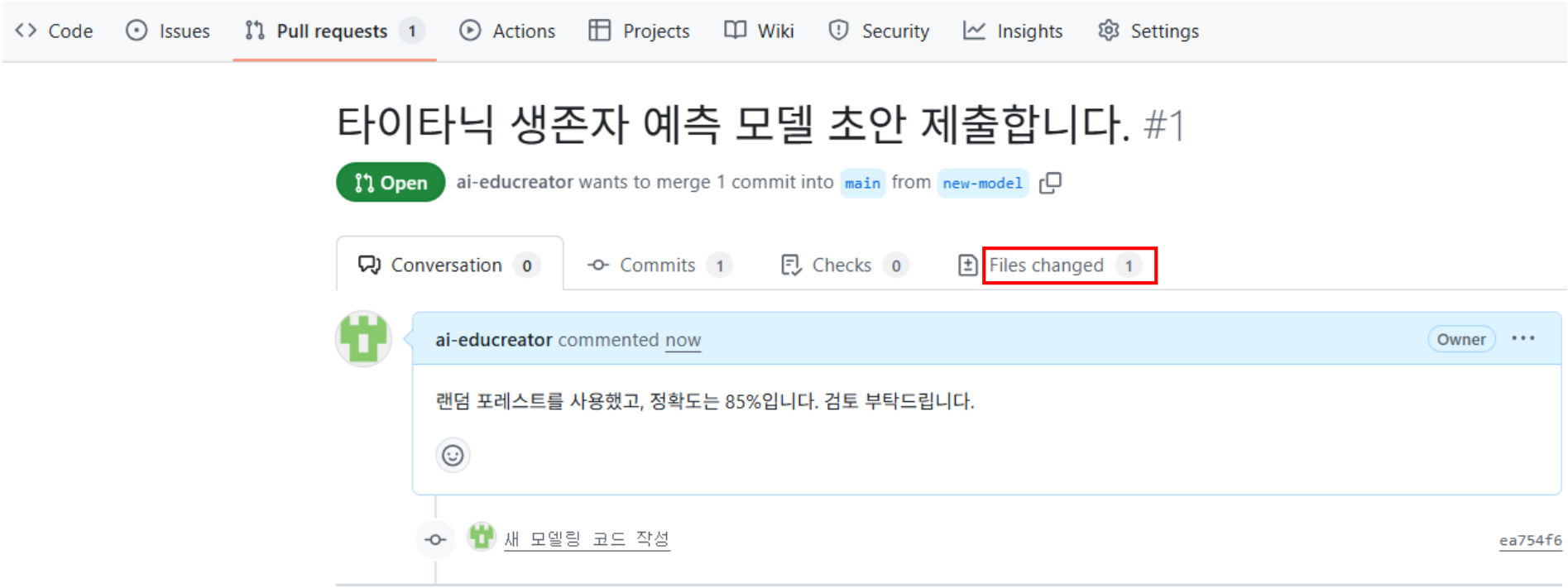


github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

• 5단계: 병합 (Merge) → Files changed 탭 클릭



github를 이용한 코드 저장 및 버전 관리

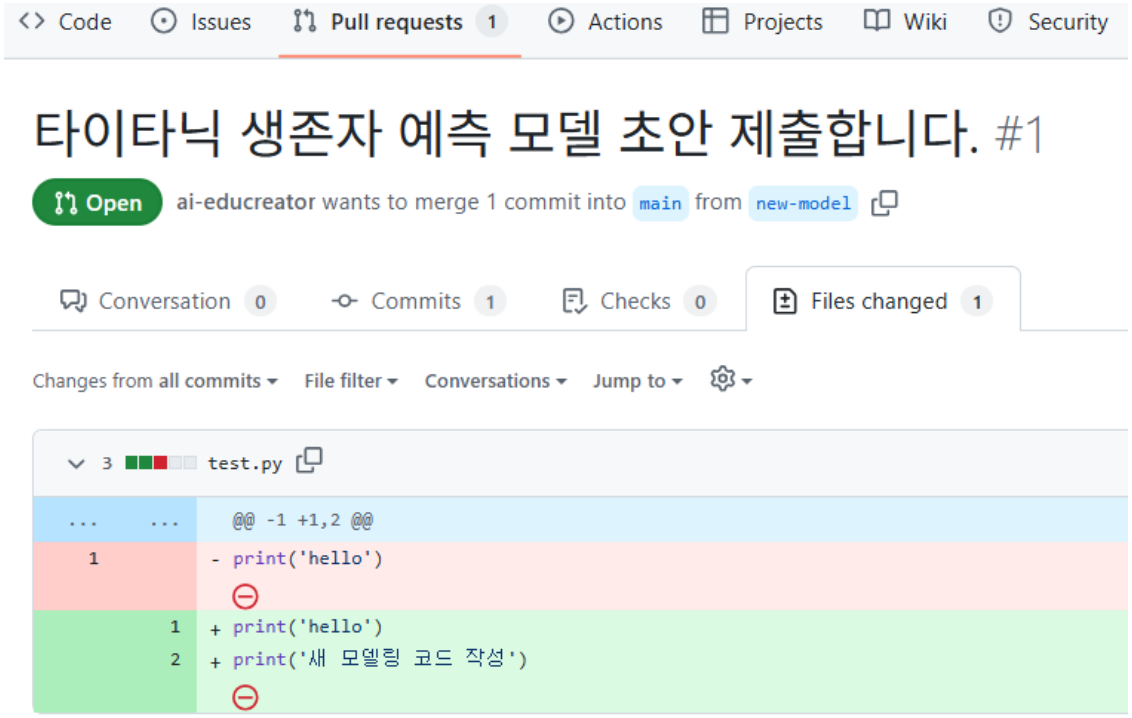
14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 5단계: 변경된 코드 확인

1. 빨간색 배경: 삭제된 코드 / 초록색 배경: 새로 추가된 코드

2. 이 화면을 보며 "코드가 효율적인지", "오타는 없는지" 꼼꼼히 확인

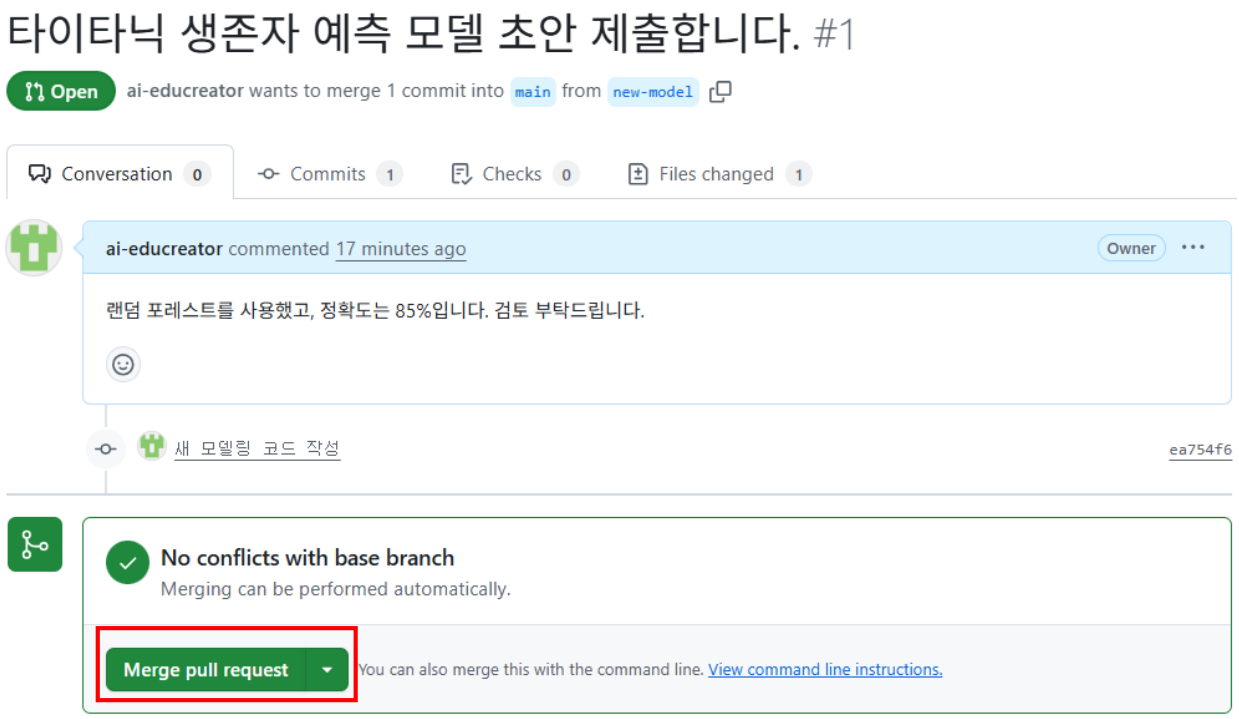


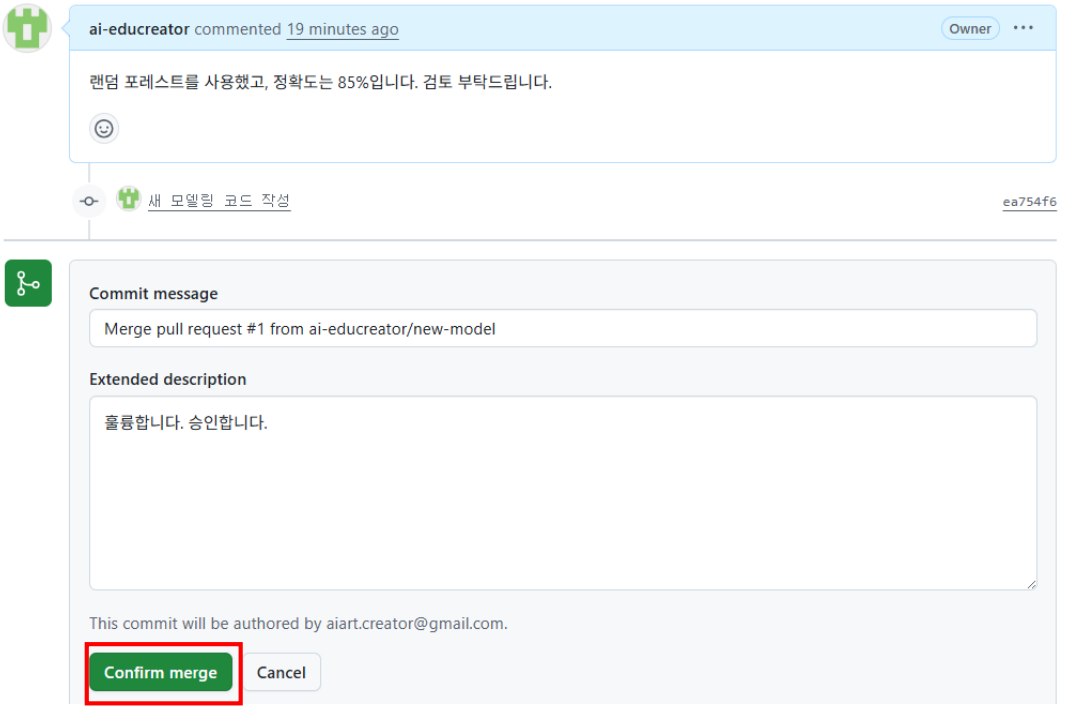
github를 이용한 코드 저장 및 버전 관리

14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 5단계: Conversation 탭 클릭 → 아래 연두색 Merge pull request 클릭 → Confirm merge 클릭





github를 이용한 코드 저장 및 버전 관리

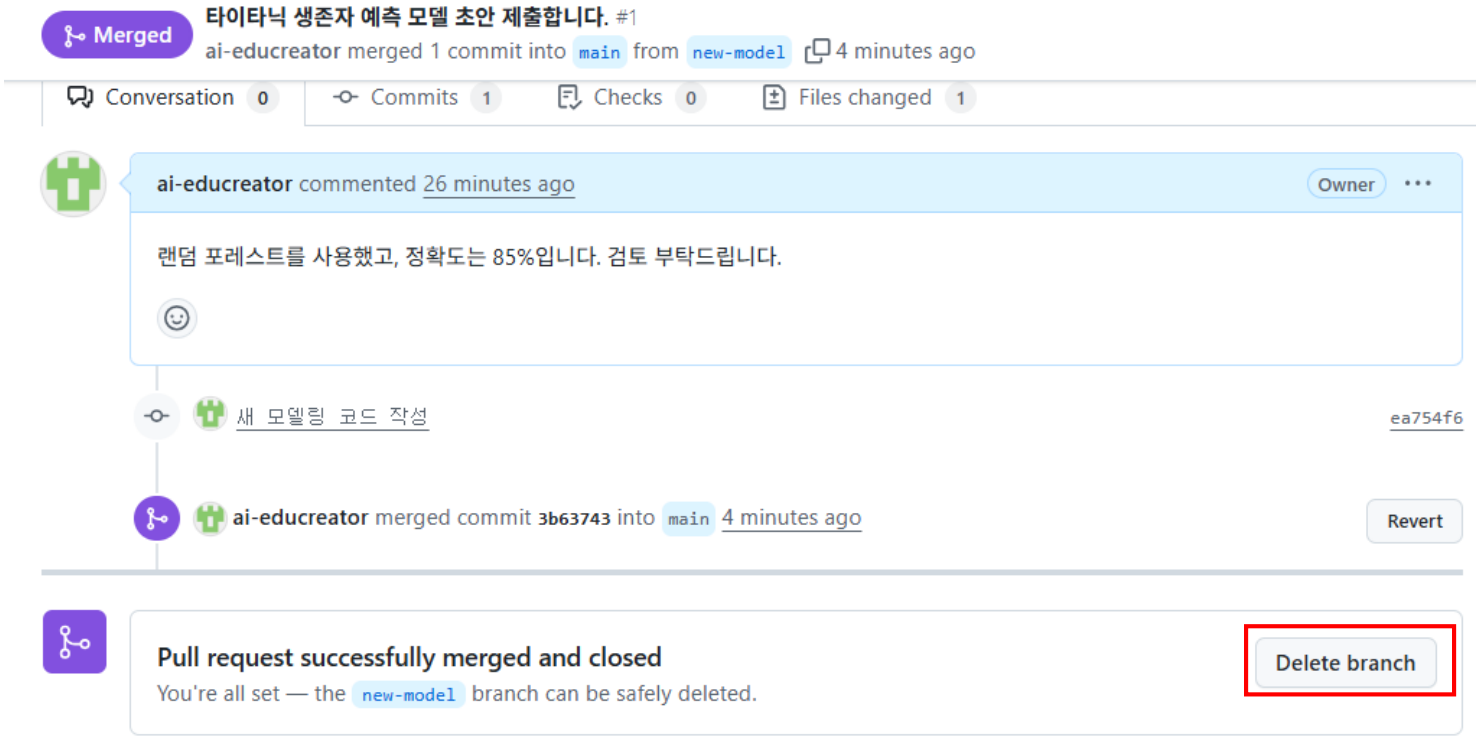
14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

③ 실습

- 정리:

1. 상태 변화: 아이콘이 '보라색(Merged)'으로 바뀌면 합치기 성공

2. Delete branch 버튼 클릭 → 이미 합쳐진(용도가 끝난) 작업용 브랜치는 지워주는 것이 깔끔함



github를 이용한 코드 저장 및 버전 관리

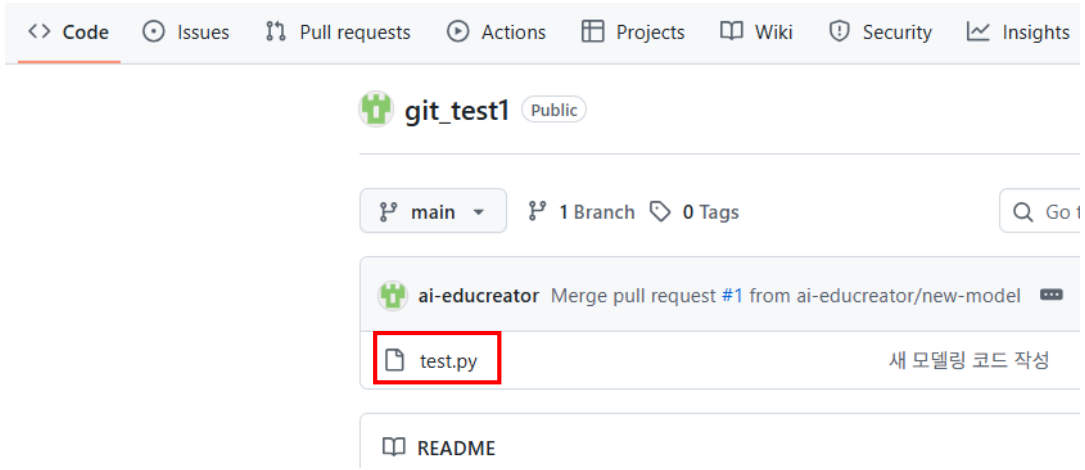
14) '팀 프로젝트' 를 위한 'GitHub Flow' 전략

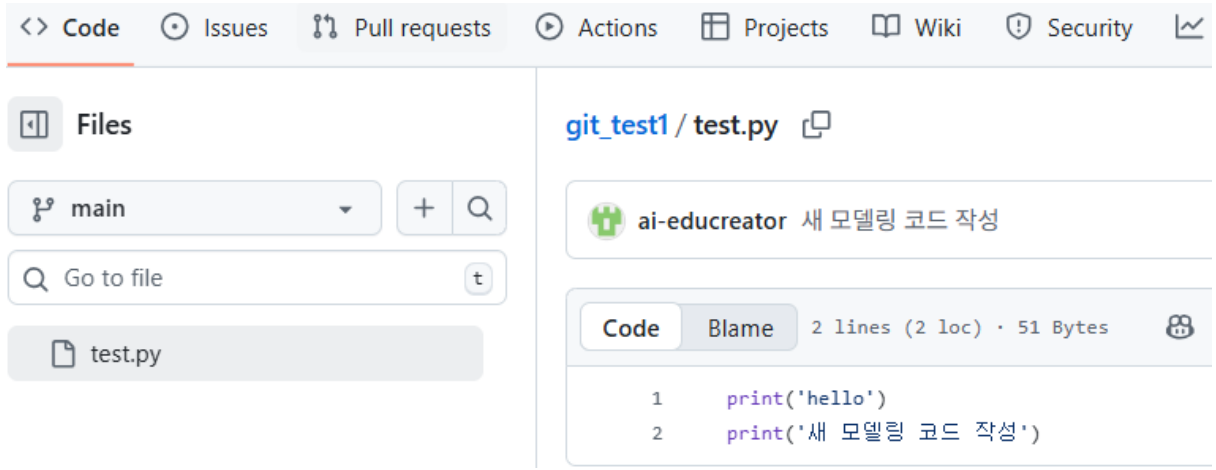
③ 실습

- 최종 확인

1. GitHub 저장소의 <> Code 탭(메인 화면) 클릭

2. main 브랜치의 파일 클릭 → 파일 내용이 내가 수정한 코드로 바뀌어 있는지 확인





github를 이용한 코드 저장 및 버전 관리

15) **'충돌(Conflict)'** 현상 해결

① 상황 예시

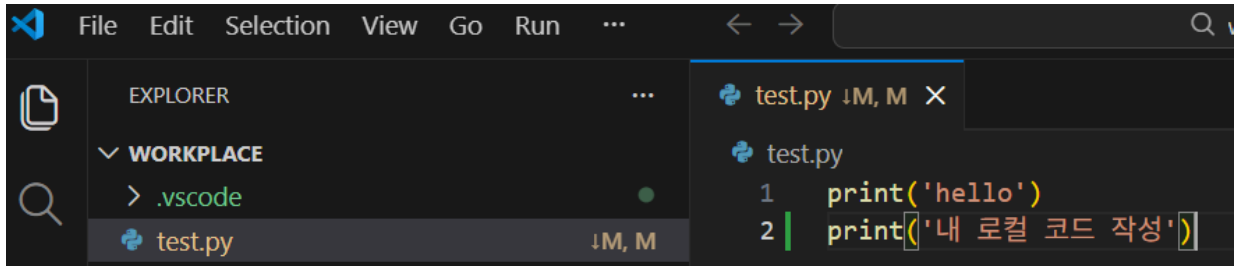
- [GitHub]: 두 번째 줄에 print('새 모델링 코드 작성') 코드를 추가해서 합쳐진 상태
- [내 컴퓨터]: 나는 두 파일이 합쳐진 상태를 모르고 두 번째 줄에 print('내 로컬 코드 작성') 작성
- 상황 발생: 내가 두 번째 줄에 '완전히 다른 내용'을 작성해서 **Push**
- 결과: 에러 발생! (Rejected) 발생
- 에러 발생 원인: 내 컴퓨터가 GitHub의 변화를 반영하고 있지 못한 상태

github를 이용한 코드 저장 및 버전 관리

15) **'충돌(Conflict)'** 현상 해결

② 상황 예시에 대한 실습

- VS Code 좌측 하단에 브랜치 이름이 main 으로 되어 있는지 확인
- test.py의 2번째 줄에 print('내 로컬 코드 작성') 작성 및 저장
  (GitHub의 내용과 다르게 작성하는 것이 핵심!)
- 상황 발생: 내가 두 번째 줄에 '완전히 다른 내용'을 작성해서 Push
- 결과: 에러 발생! (Rejected)



github를 이용한 코드 저장 및 버전 관리

15) **'충돌(Conflict)'** 현상 해결

② 상황 예시에 대한 실습

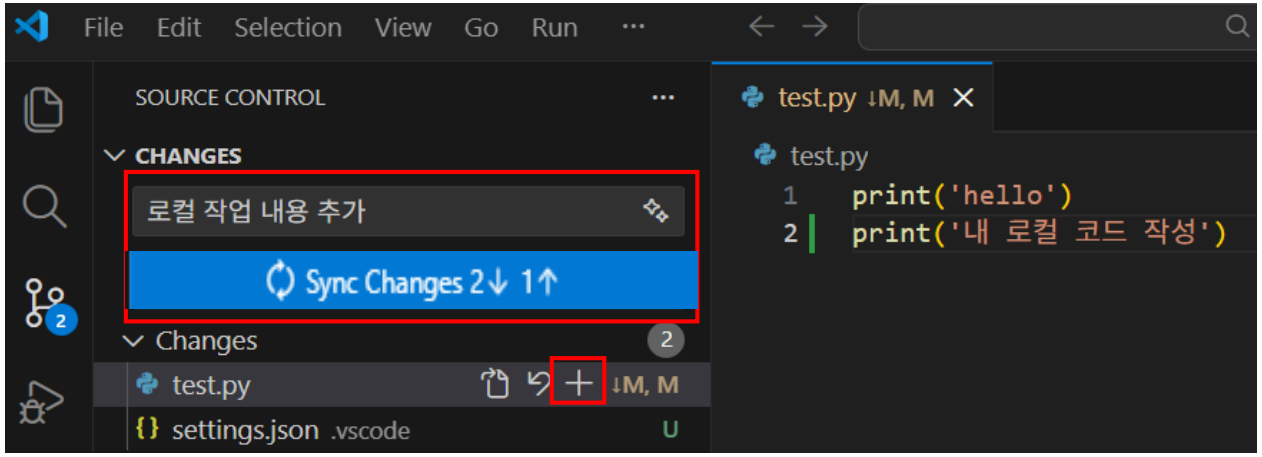
- 커밋 (Commit)

1. 코드 작성 및 저장

2. Source Control 탭 → test.py → + 버튼 (Staging)

3. 메시지: 로컬 작업 내용 추가

4. Commit 버튼 클릭



github를 이용한 코드 저장 및 버전 관리

15) **'충돌(Conflict)'** 현상 해결

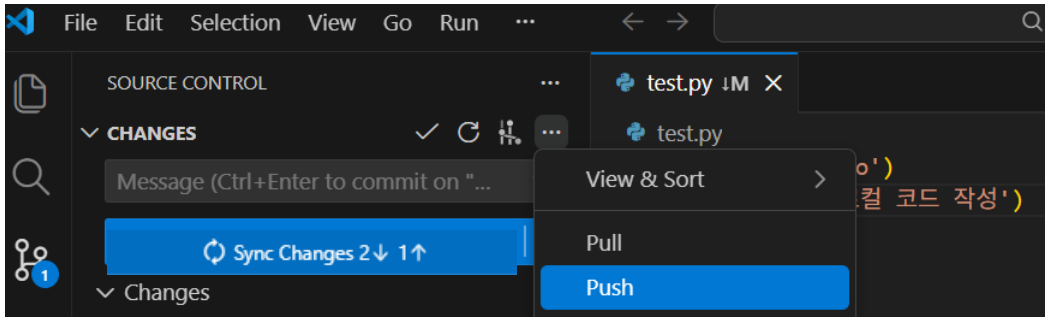
② 상황 예시에 대한 실습

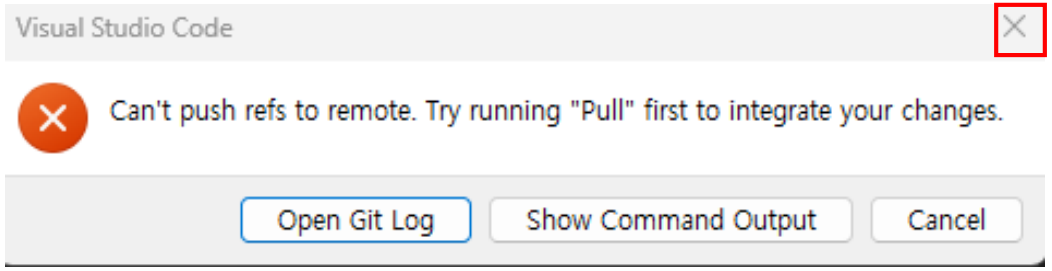
- Push 시도

1. Push 클릭

2. 에러 발생! → 버전 차이 발생

3. 팝업 창 우측 상단의 [X] 버튼을 눌러 팝업 창 닫기





github를 이용한 코드 저장 및 버전 관리

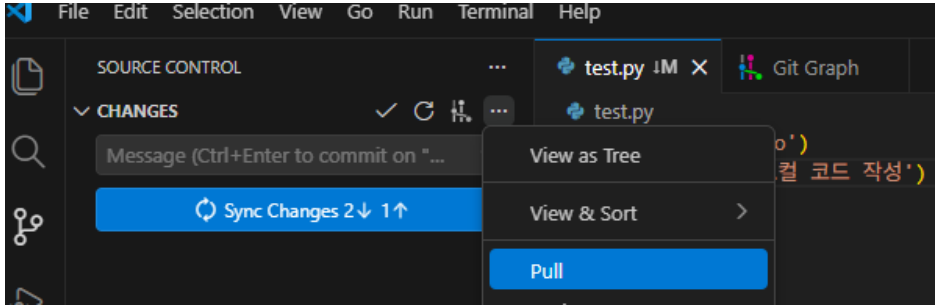
15) **'충돌(Conflict)'** 현상 해결

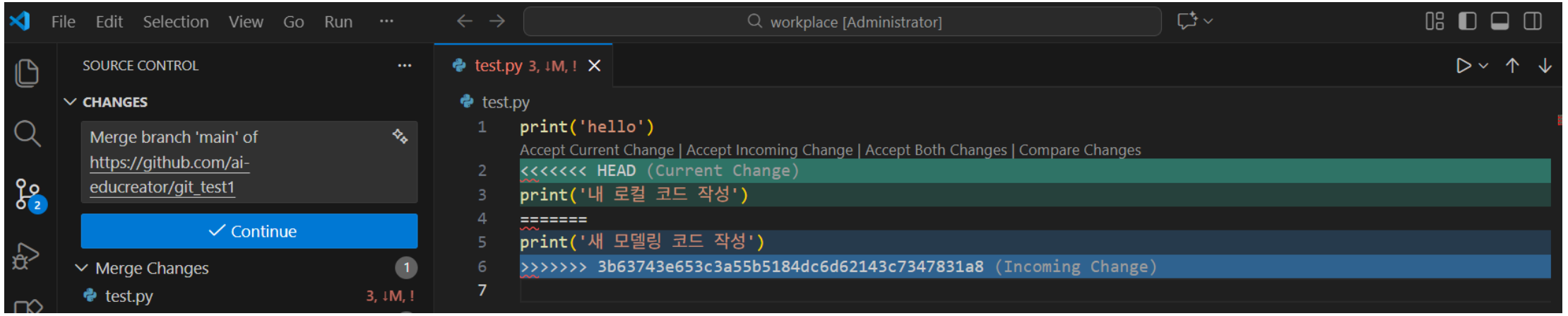
③ 상황 예시에 대한 실습

- **Pull 실행: Source Control 탭의 Changes ... 메뉴 -> Pull 버튼 클릭**

1. "Conflict!" 발생 → VS Code 하단에 "Conflict!" 알림 발생

2. 파일 탐색기에서 test.py 옆에 ! (느낌표) 아이콘이 뜨고 파일명이 빨간색으로 변함





github를 이용한 코드 저장 및 버전 관리

15) **'충돌(Conflict)'** 현상 해결

④ Merge Editor로 해결하기

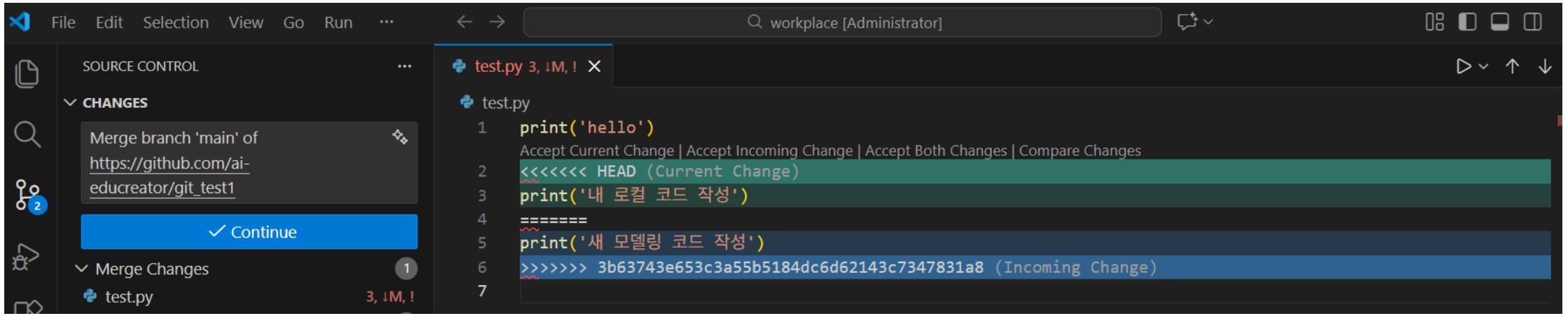
- 화면 분석

  # 1. <<<<<<< HEAD (Current Change): 내 코드 (print('내 로컬 코드 작성'))

  # 2. =======: 중앙선

  # 3. >>>>>>> ... (Incoming Change): GitHub 코드 (print('새 모델링 코드 작성'))

- **Git의 요청: "2번째 줄에 무엇을 남길까? 네 것? 아니면 GitHub 것?"**



github를 이용한 코드 저장 및 버전 관리

15) **'충돌(Conflict)'** 현상 해결

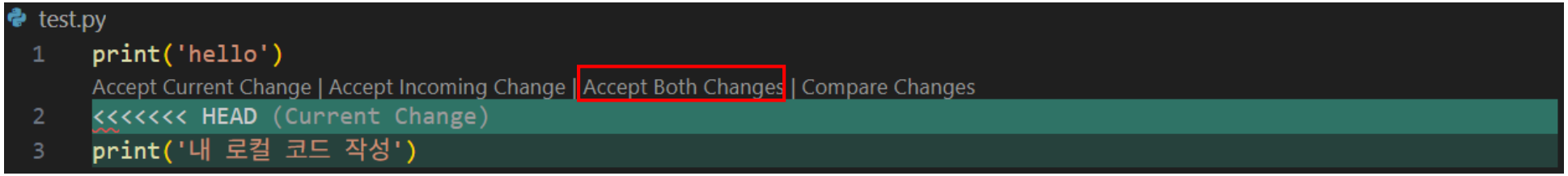
④ Merge Editor로 해결하기

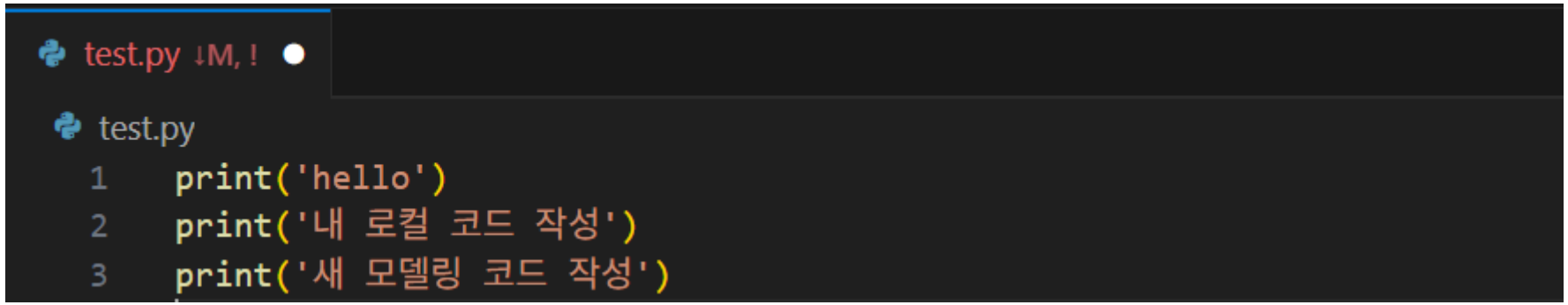
- 해결 (옵션 선택)

  # 1. 코드 위에 나타난 회색 글씨 중 [Accept Both Changes] (두 변경 사항 모두 수락)를 클릭

  # 2. 상황: 팀원의 코드도 살리고, 내 코드도 살려야 할 경우

  # 3. 결과: 2번째, 3번째 줄에 두 문장이 나란히 남는다.





github를 이용한 코드 저장 및 버전 관리

15) **'충돌(Conflict)'** 현상 해결

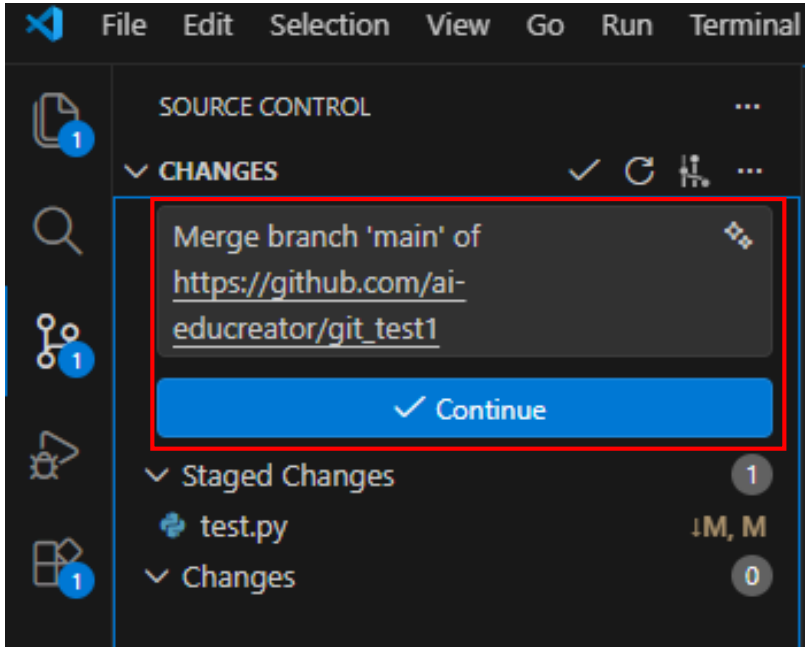
⑤ 해결 완료 및 업로드

- 병합 커밋 (Merge Commit)

1. Source Control 탭으로 이동

2. test.py → + 버튼 클릭

3. 메시지 입력창에 자동으로 생성된 Merge branch... 확인 후 Cotinue 버튼 클릭



github를 이용한 코드 저장 및 버전 관리

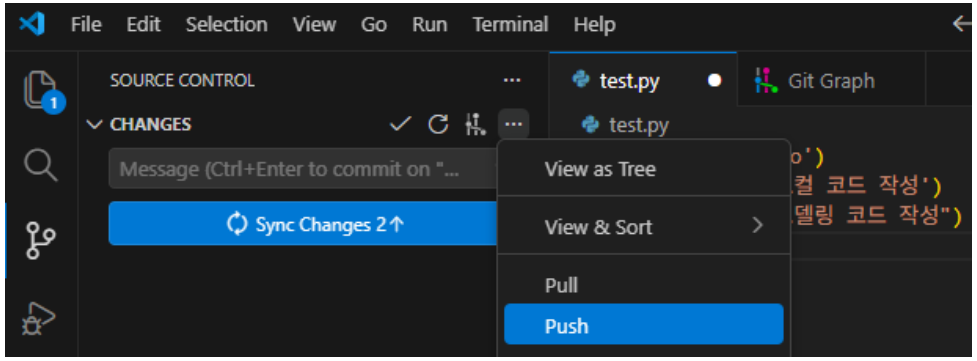
15) **'충돌(Conflict)'** 현상 해결

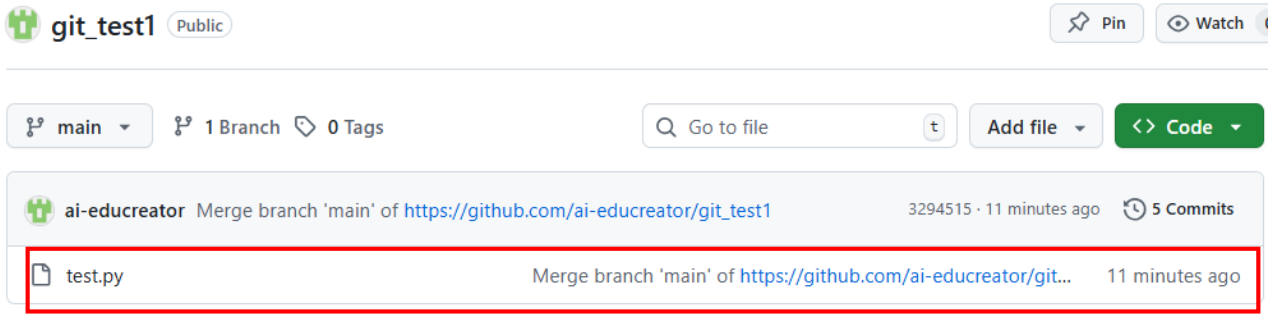
⑤ 해결 완료 및 업로드

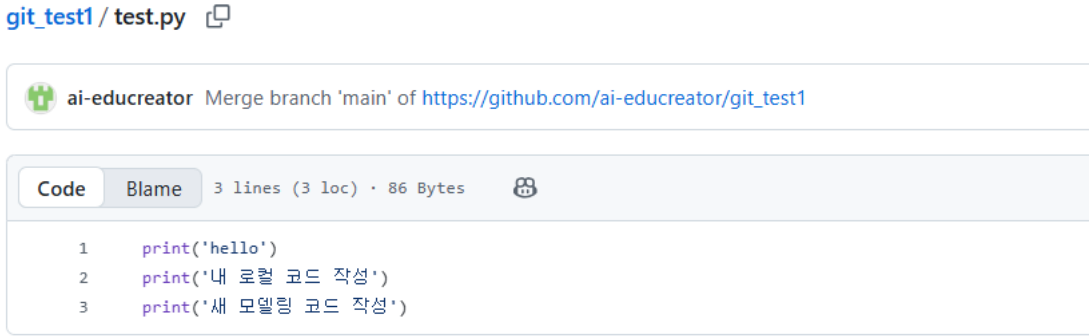
- 최종 Push

1. Push버튼을 눌러 GitHub에 업로드

2. 성공 확인: 에러 알림 없이 업로드되고, GitHub 웹사이트에서 두 문장이 모두 보이는지 확인







github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

① 'GitHub Flow' 전략이란?

- 기능: 팀 프로젝트 시 서로의 코드를 덮어쓰거나 망가뜨리지 않고, 안전하게 각자의 작업을 진행한 뒤 검토를 거쳐 합치게 해준다
- 핵심 전략: "작업은 Branch에서, 합치는 건 GitHub에서."
- 'GitHub Flow ' 전략 5단계

1. Issue: 할 일 정의

2. Branch: 작업 전용 공간 생성 (Main 아님!)

3. Commit & Push: 로컬에서 코드 작성 후 업로드

4. Pull requests: 검토(결재) 요청

5. Review & Merge: 승인 및 main에 병합

github를 이용한 코드 저장 및 버전 관리

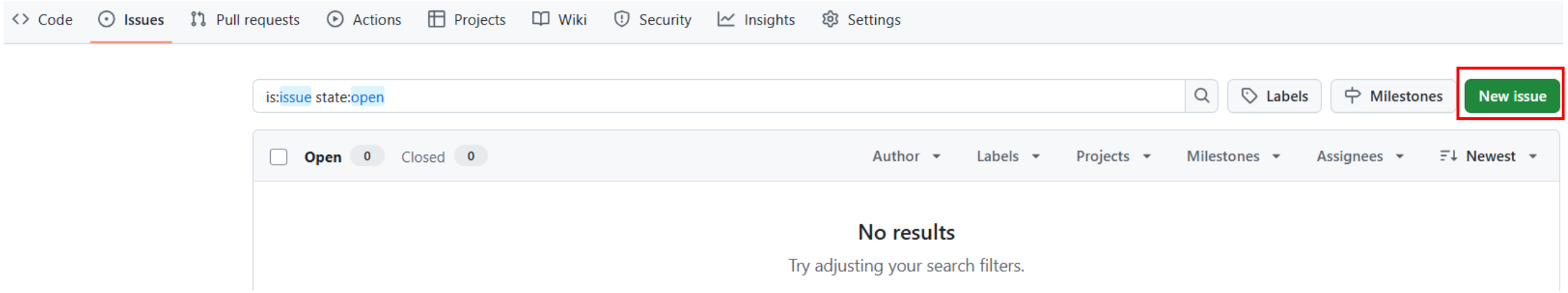
16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

- 시나리오: "팀장님이 당신에게 타이타닉 데이터에서 성별(Sex)에 따른 생존율을 분석해달라는 업무를 지시했습니다."
- [Step 1] Issue 등록하기

  # 1. GitHub 저장소 상단 [Issues] 탭 클릭

  # 2. 초록색 [New issue] 버튼 클릭



github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

3. 내용 작성

- Title: 성별 생존율 분석 기능 추가
- Description

```
## 목적: 성별(Sex)이 생존에 미치는 영향 분석
## 할 일
 - [ ] groupby를 이용한 평균 생존율 계산
 - [ ] 결과 출력 코드 작성
```

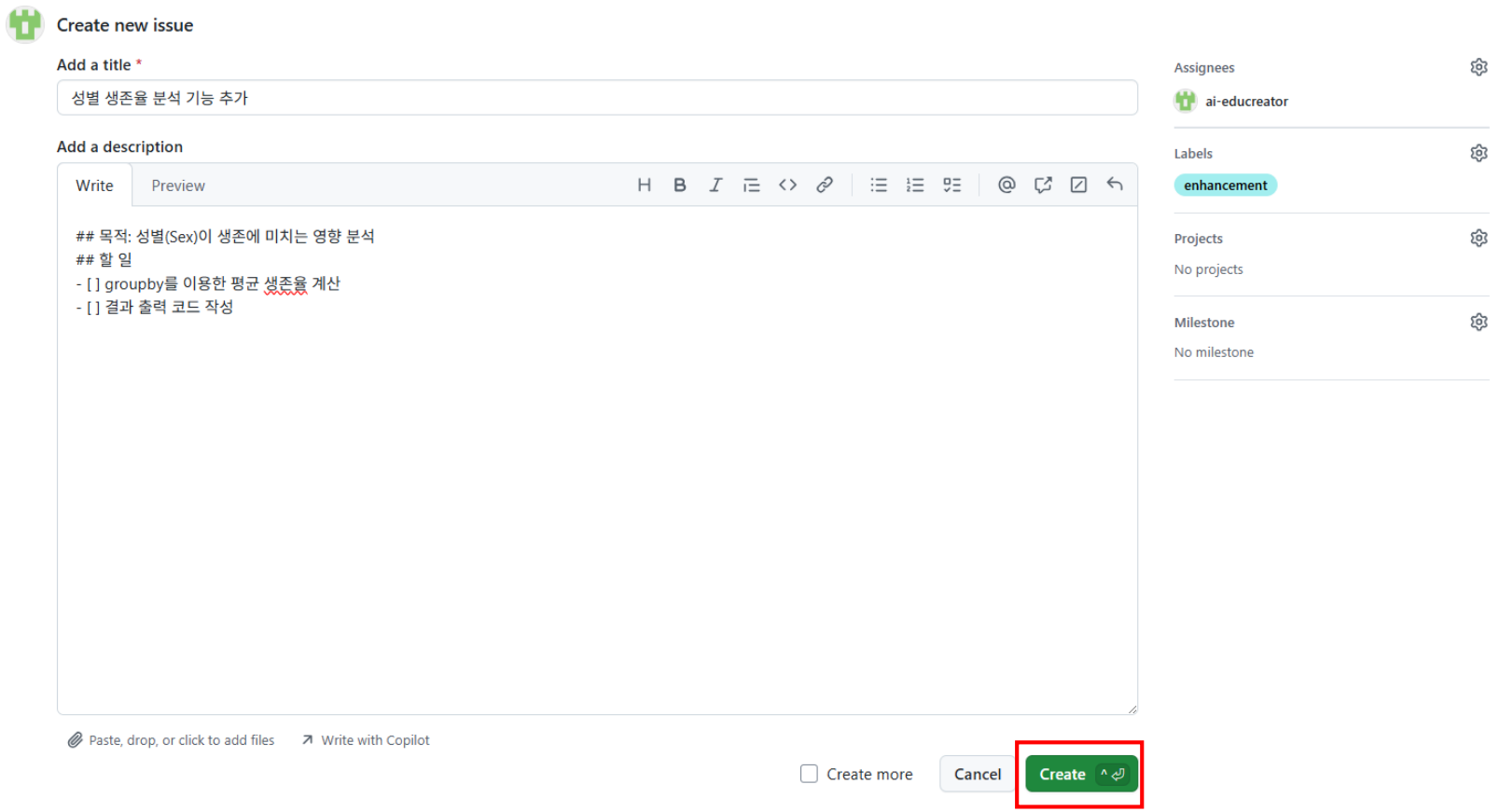
- Assignees(우측 사이드바): 톱니바퀴를 눌러 '나 자신(본인 계정)'을 선택
- Labels (우측 사이드바): 톱니바퀴를 눌러 enhancement (기능 추가) 선택

github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

3. 내용 작성



github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

4. 우측 하단의 초록색 [Create] 버튼 클릭 → 제목 옆에 붙은 #2 (이슈 번호)를 반드시 기억할 것



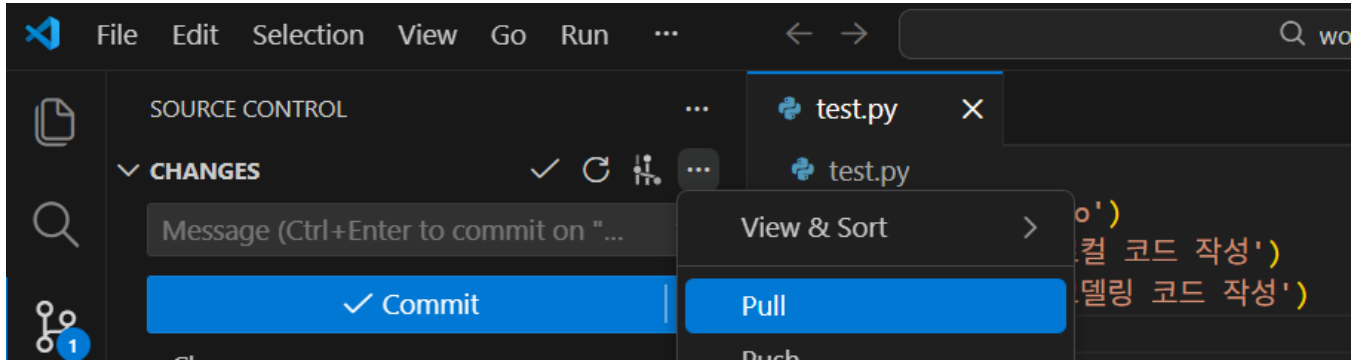
github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 2] Branch 생성 및 작업 (개발)

1. 최신 버전으로의 동기화 (필수): main 브랜치 상태에서 Pull 클릭



github를 이용한 코드 저장 및 버전 관리

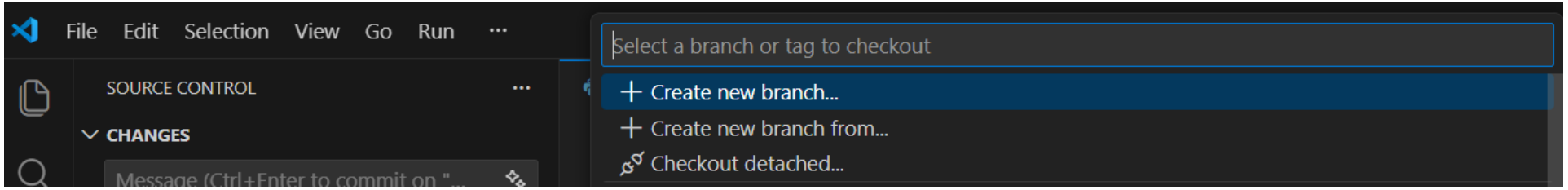
16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

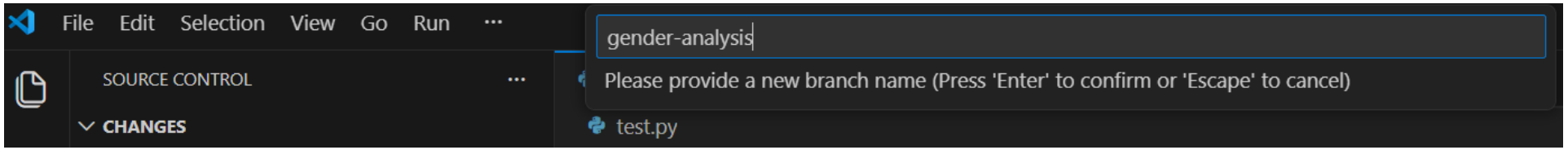
② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 2] Branch 생성 및 작업 (개발)

2. 브랜치 생성

- VS Code 좌측 하단 main 클릭 → Create new branch.
- gender-analysis 입력 (직관적인 이름 사용)





github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 2] Branch 생성 및 작업 (개발)

3. 코드 작성

- # 성별 생존율 분석, 실제 분석 로직 대신 print문으로 대체
- print("Female Survival Rate: 0.74")
- print("Male Survival Rate: 0.18")

test.py M

test.py

```
1  print('hello')
2  print('내 로컬 코드 작성')
3  print('새 모델링 코드 작성')
4
5  # 성별 생존율 분석, 실제 분석 로직 대신 print문으로 대체
6  print("Female Survival Rate: 0.74")
7  print("Male Survival Rate: 0.18")
```

github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

- [Step 3] Commit & Push

1. Source Control 탭으로 이동 → test.py → + 버튼 클릭

2. Commit 메시지 작성 (핵심)

- 성별 생존율 분석 코드 추가 (Resolves #2)
- 설명: Resolves #번호를 적으면, 나중에 병합될 때 해당 이슈가 자동으로 '완료(Closed)' 처리 됨

3. Push 버튼 클릭 → GitHub에 업로드

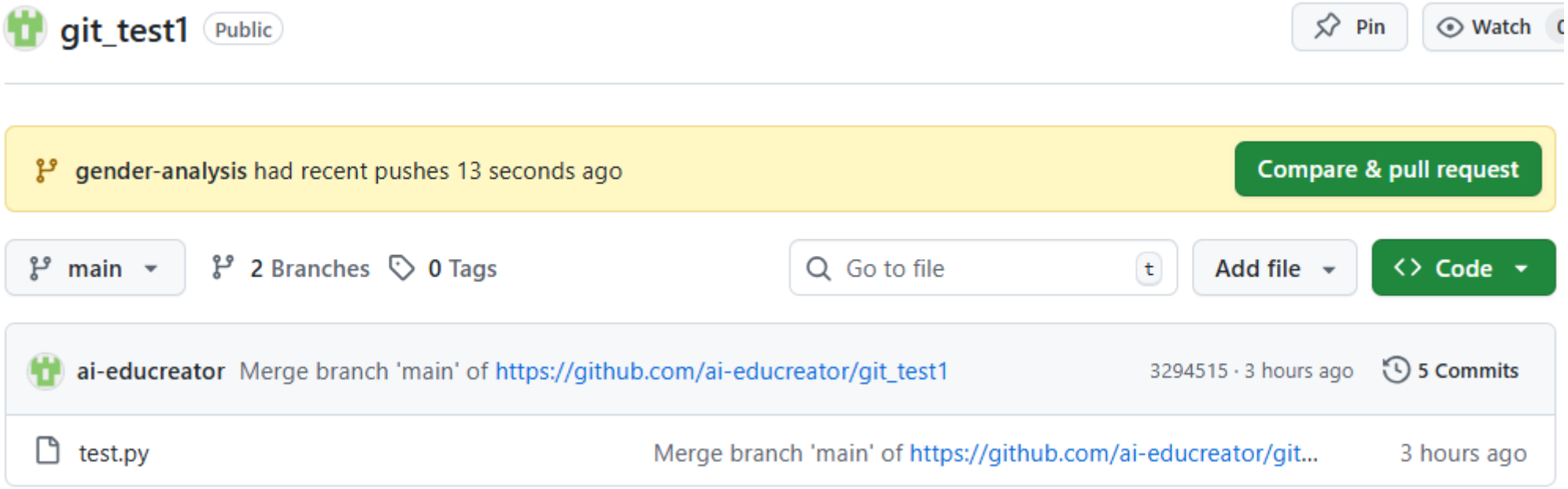
github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 4] Pull requests 작성 (결재 요청)

1. GitHub 저장소 페이지(<> Code 탭)로 이동



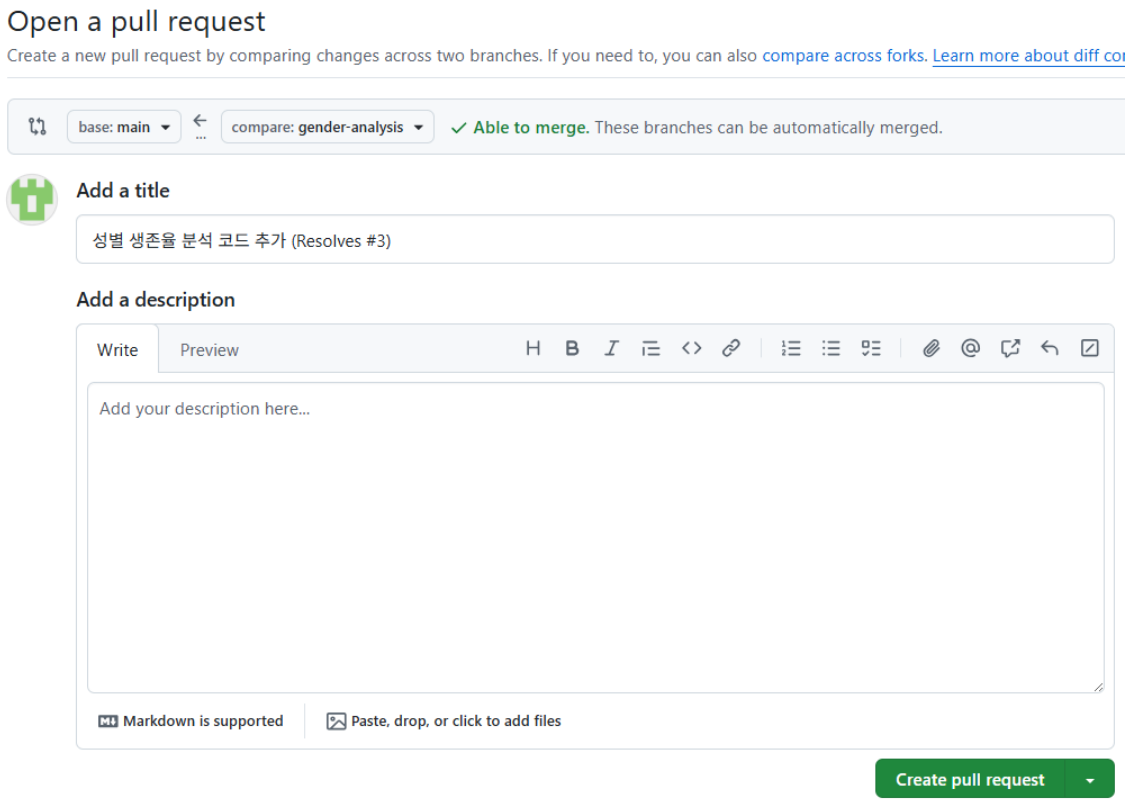
github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

- [Step 4] Pull requests 작성 (결재 요청)

  # 2. 상단에 뜬 노란색 박스의 'Compare & pull request' 버튼 클릭



github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 4] Pull requests 작성 (결재 요청)

3. Pull requests 신청서 작성

- 제목: 성별 생존율 분석 로직 구현 (Issue #2)
- 내용: "요청하신 분석 로직 구현 완료했습니다. 여성이 남성보다 생존율이 높습니다."
- Reviewers (우측 사이드바): 팀 동료 선택

→ 주의: 목록에 아무도 안 뜬다면? -> 먼저 Settings > Collaborators에서 특정인 초대

→ [혼자 실습하는 경우('우리'의 상황]

비워두고 넘어간다 (Reviewer 없이도 PR 생성 가능)

- 'Create pull request' 녹색 버튼 클릭

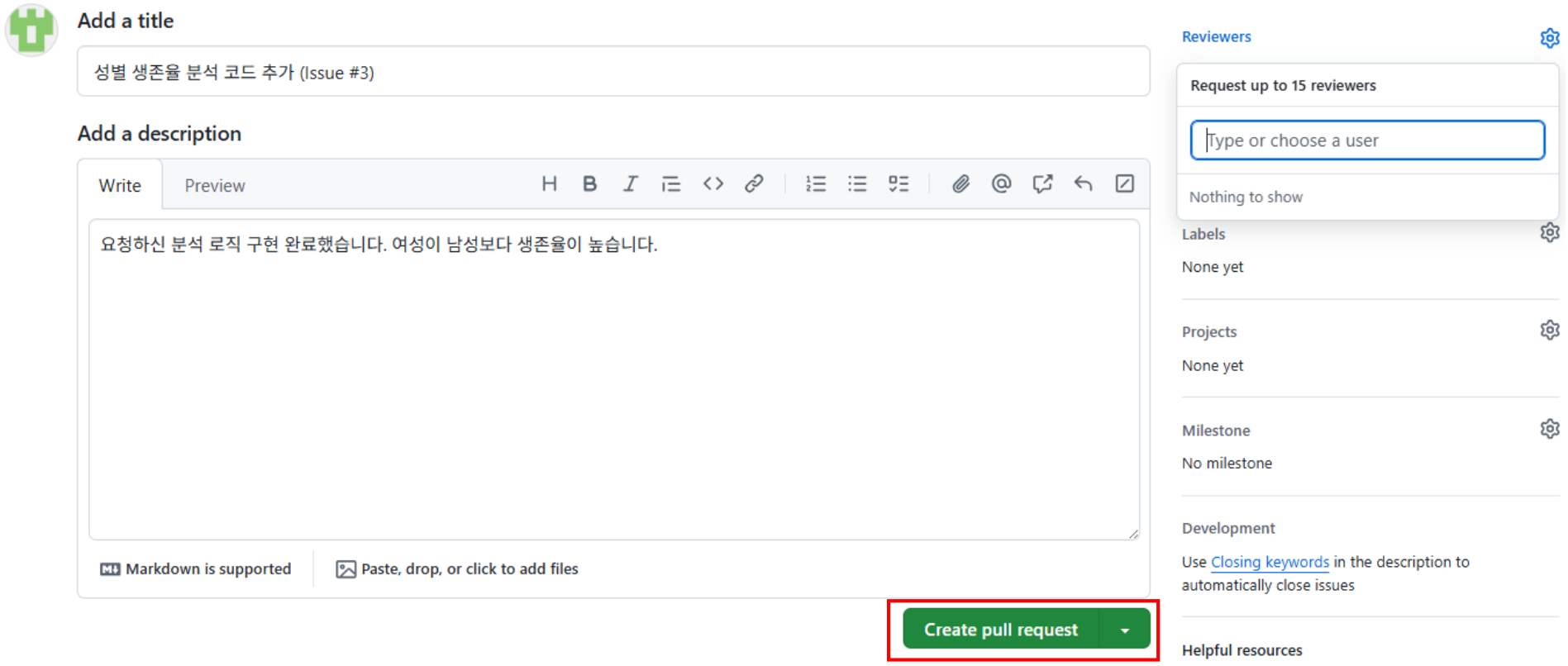
github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 4] Pull requests 작성 (결재 요청)

3. Pull requests 신청서 작성



github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

- [Step 5] 자가 점검 (Self-Review) - "내가 나의 팀장이 된다"

  # 1. 혼자 개발하더라도 '검토' 과정은 필수 → 바로 합치지 말고 코드를 눈으로 확인할 것!

  # 2. Pull requests 화면 상단의 'Files changed' 탭 클릭

  # 3. 변경 내역 검증:

  - 초록색 배경: 내가 추가한 코드가 맞는지 확인
  - 빨간색 배경: 혹시 실수로 지워진 코드는 없는지 확인
  - 보충: 실무에서는 이 단계에서 print() 같은 디버깅용 코드를 실수로 남기지 않았는지 확인

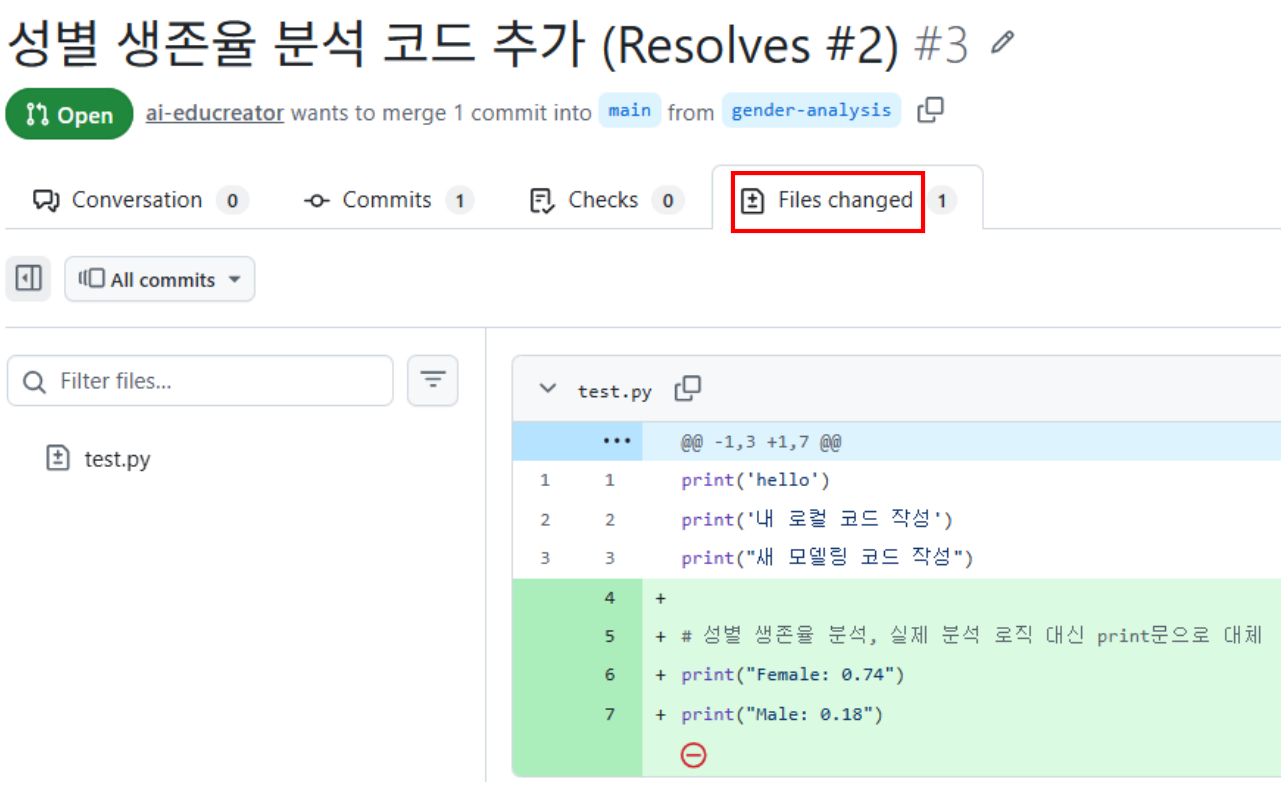
  # 4. Conversation 탭 클릭: 코드가 완벽하다면, 첫 번째 탭인 'Conversation'을 눌러 메인 화면으로 복귀

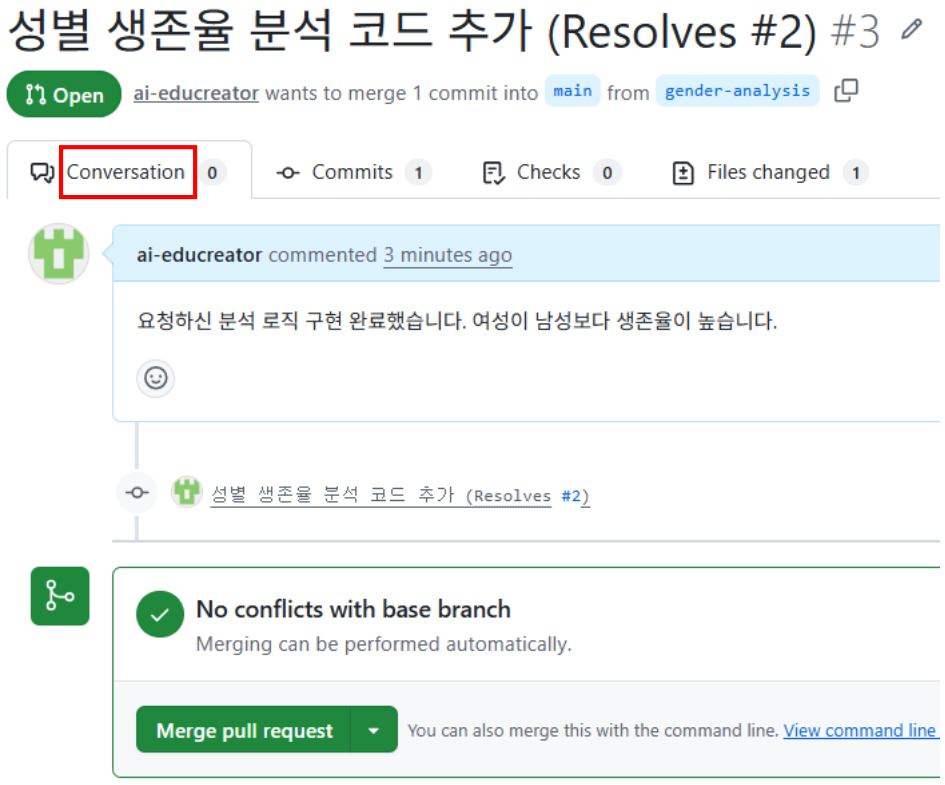
github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 5] 자가 점검 (Self-Review) - "내가 나의 팀장이 된다"





github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

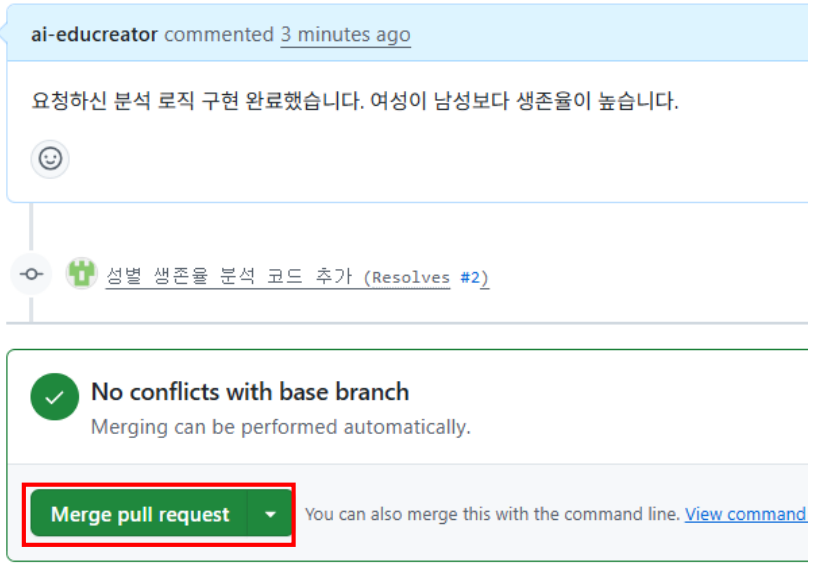
• [Step 6] 병합(Merge) 및 이슈 자동 종료

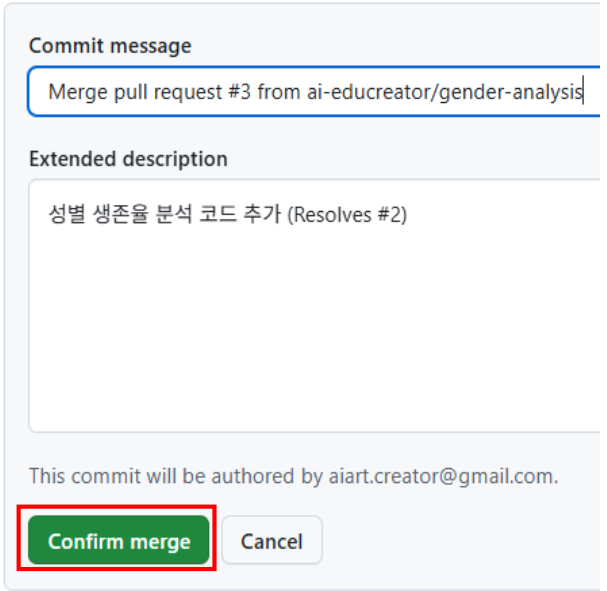
1. 검토가 끝났으니, 브랜치(gender-analysis)를 원본(main)에 공식적으로 병합

2. 병합 (Merge)

- 화면 하단의 녹색 'Merge pull request' 버튼 클릭

- 이어서 나오는 'Confirm merge' 버튼 클릭





github를 이용한 코드 저장 및 버전 관리

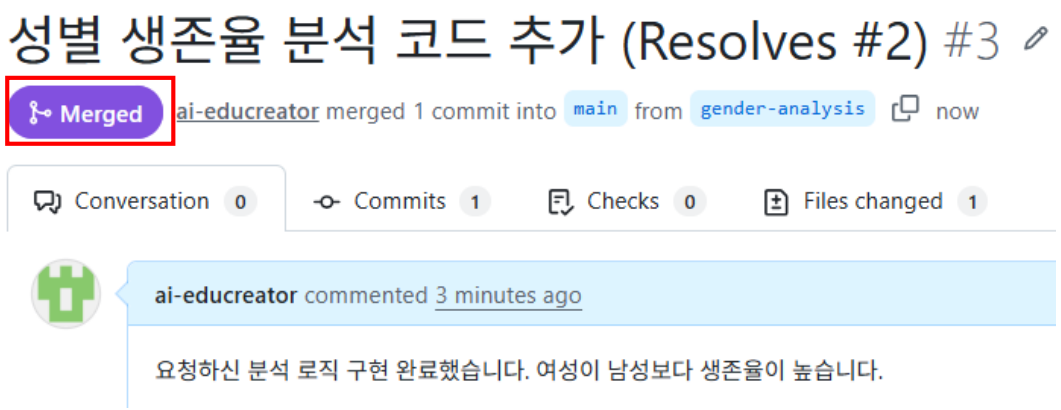
16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

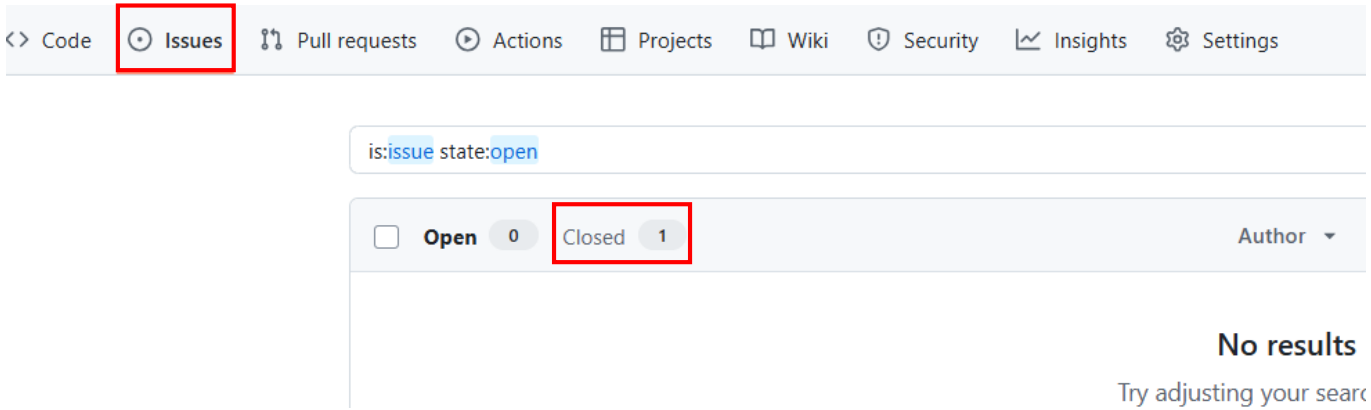
② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 6] 병합(Merge) 및 이슈 자동 종료

3. 성공 확인 (가장 중요):

- 상태 아이콘: 상단 아이콘이 보라색 Merged로 변했는지 확인.
- 이슈 상태: 상단 메뉴의 'Issues' 탭 클릭 → 아까 만든 #2 이슈가 사라졌는지(Closed) 확인





github를 이용한 코드 저장 및 버전 관리

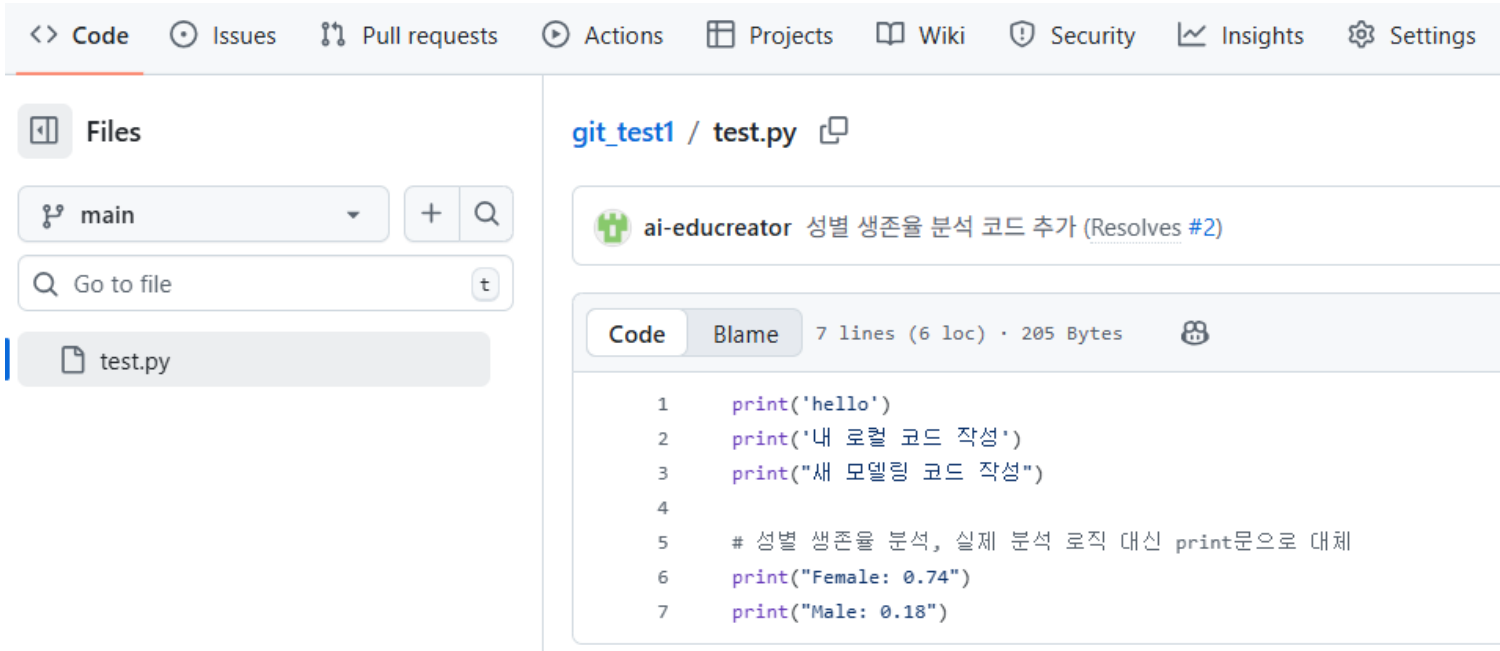
16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 6] 병합(Merge) 및 이슈 자동 종료

4. 결과 확인

- GitHub 저장소 페이지(<> Code 탭)로 이동
- test.py 파일 클릭 → 코드 확인



github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

- [Step 7] 로컬 동기화 (Sync) - 프로젝트 마무리

  # 1. GitHub는 최신 상태가 되었지만, 내 컴퓨터(VS Code)는 아직 옛날 상태

  # 2. VS Code 좌측 하단 브랜치를 클릭하여 gender-analysis에서 main 으로 변경

  # 3. 동기화 (Pull): Source Control 탭의 Pull 버튼 클릭

  # 4. 최종 결과 확인

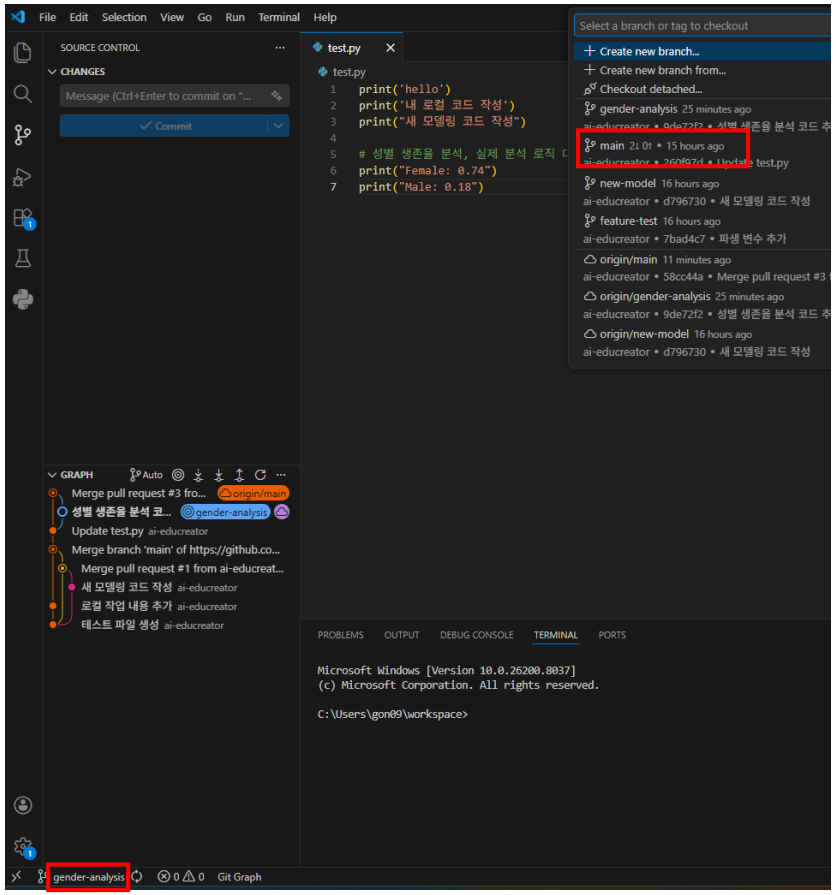
  - 파일 탐색기에서 test.py 클릭
  - 브랜치에서 작성했던 코드가 main 파일에도 안전하게 들어와 있다면 성공

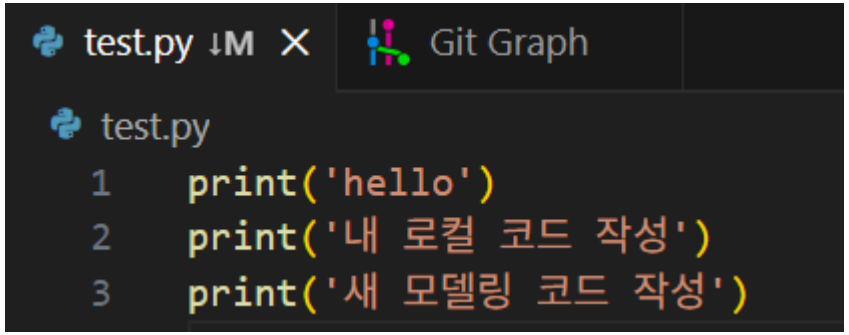
github를 이용한 코드 저장 및 버전 관리

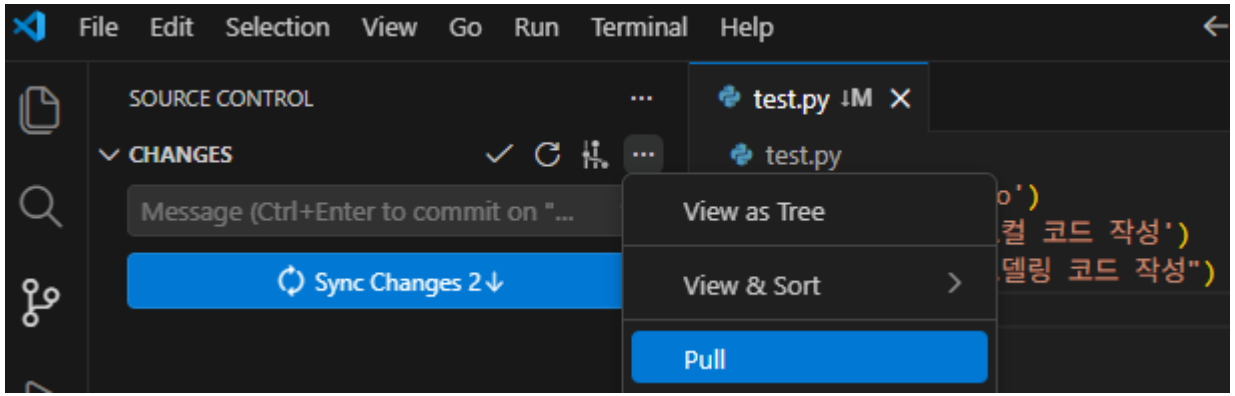
16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 7] 로컬 동기화 (Sync) - 프로젝트 마무리





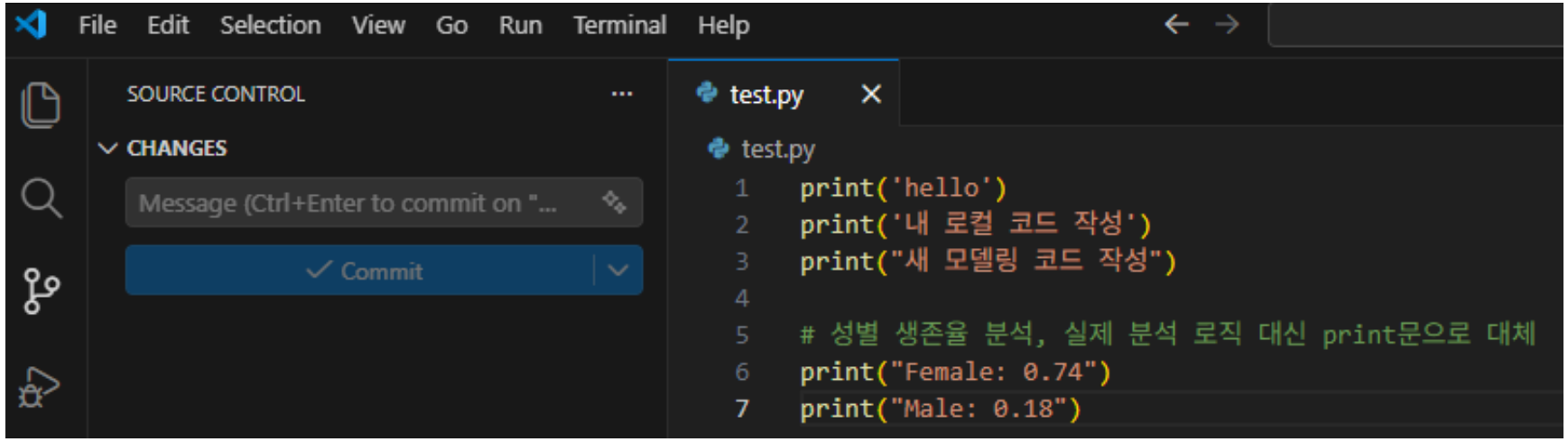


github를 이용한 코드 저장 및 버전 관리

16) '팀 프로젝트'을 위한 'GitHub Flow' 전략: Issue 추가 및 정리

② 실습 예시: 타이타닉 생존자 성별 분석 프로젝트

• [Step 7] 로컬 동기화 (Sync) - 프로젝트 마무리



핵심 요약: 팀 프로젝트 협업 무기 Git & GitHub

1) 버전 관리

① Commit: 단순한 저장이 아니라, 나의 개발 과정을 증명하는 '역사적 기록'이다.

② Sync (Push & Pull): 내 컴퓨터(Local)와 GitHub(Remote)의 '시차'를 맞추는 습관이 사고를 예방한다.

2) 협업의 기술

① Branch: 원본 데이터(main)를 보호하고, '복사본 공간'에서 마음껏 실험한다.

② Conflict: 충돌은 에러가 아니다. 더 나은 코드를 선택하기 위한 '조율 과정'이다.

3) 프로의 방식 (GitHub Flow)

① Issue: 말로 하지 말고 업무를 정의한다.

② Pull requests(PR): 코드를 합치기 전, 동료에게 검토 받는 '결재 서류'이자 '성장 기회'다.


MISSION:
COMPLETE